



RAPPORT DE STAGE

En vue de l'obtention du diplôme de Licence en Business Computing

Parcours Business Intelligence

Réalisé par : **Oussema Korbosli**

**« Conception et développement d'un système intelligent de gestion des rôles
et de pointage des employés avec tableau de bord décisionnel »**

Du 16 février au 16 juin 2026

Maître de stage : Mr. Melek Aziz Elmoula

Encadrant académique : Mr. Heni Abidi

Année universitaire 2025/2026

	<i>Nom & Prénom</i>	<i>Date et Signature</i>
Maître de stage	Mr. Melek Aziz Elmoula	28/07/2026 
Encadrant académique	Mr. Heni Abidi	

Résumé

Réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne**, ce projet de fin d'études porte sur la conception et le développement d'un **système intelligent de gestion des rôles et de pointage des employés**, doté d'un tableau de bord décisionnel (Business Intelligence). L'application centralise la gestion des agences, des employés, des clients et des objectifs commerciaux, gère les rôles et la sécurité des accès, assure le **pointage (présence) des employés**, et formalise les opérations sensibles au moyen de circuits de validation. Développée avec la plateforme **Jakarta EE 10** (Servlets, JSP, EJB, JPA/Hibernate) au-dessus d'une base **Oracle**, elle est complétée par un volet décisionnel : une chaîne **ETL** (Python) consolide les données dans un datamart qui alimente des vues exposées à **Microsoft Power BI**, un tableau de bord embarqué (effectifs, présence, production) et une **segmentation des agences** par apprentissage non supervisé (clustering). La solution améliore la centralisation des données, la traçabilité des opérations et l'aide à la décision.

Mots-clés : gestion des rôles, pointage des employés, Business Intelligence, ETL, aide à la décision, apprentissage automatique, clustering, Jakarta EE, Oracle, Power BI.

Abstract

Carried out within **BTK – Banque Tuniso-Koweïtienne**, this end-of-studies project deals with the design and development of an **intelligent system for managing employee roles and time-tracking (attendance)**, featuring a business-intelligence dashboard. The application centralizes the management of branches, employees, customers and commercial objectives, handles roles and access security, records **employee attendance (clock-in/clock-out)**, and formalizes sensitive operations through approval workflows. Built on the **Jakarta EE 10** platform (Servlets, JSP, EJB, JPA/Hibernate) over an **Oracle** database, it is complemented by a decision-support layer : a Python **ETL** pipeline consolidates data into a datamart that feeds analytical views exposed to **Microsoft Power BI**, an embedded dashboard (headcount, attendance, production) and a **branch segmentation** based on unsupervised machine learning (clustering). The solution improves data centralization, operation traceability and decision-making.

Key words : role management, employee time-tracking, Business Intelligence, ETL, decision support, machine learning, clustering, Jakarta EE, Oracle, Power BI.

Dédicaces

Je dédie ce modeste travail :

*À mes chers **parents**,*

pour votre amour inconditionnel, votre patience, votre soutien à tout instant et vos prières silencieuses qui m'ont toujours accompagné à chaque étape de mon parcours. Vous avez tout sacrifié pour que je puisse avancer, et aucun mot ne saurait exprimer ma gratitude et mon amour pour vous. Votre présence est ma plus grande force.

*À ma **sœur** et à mon **frère**,*

merci pour votre amour, vos encouragements et votre bienveillance. Vous avez été une source de réconfort et de motivation dans les moments difficiles. Votre présence compte énormément pour moi, et je suis fier de vous avoir à mes côtés.

*À mes **amis** et à mes **camarades**,*

pour les bons moments partagés et leur soutien tout au long de ces années d'études.

*À tous mes **enseignants**,*

pour la qualité de la formation et le savoir qu'ils ont su me transmettre.

À tous ceux qui m'ont aidé, de près ou de loin, à réaliser ce travail.

Oussema Korbosli

Remerciements

Au terme de ce projet de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie chaleureusement mon encadrant académique, **Mr. Heni Abidi**, pour ses conseils avisés, sa disponibilité et son suivi rigoureux tout au long de ce projet.

Mes remerciements s'adressent également à mon maître de stage, **Mr. Melek Aziz Elmoula**, Data Scientist au sein de la **BTK – Banque Tuniso-Koweïtienne**, pour la confiance accordée, l'accompagnement et le partage de l'expérience métier.

Je tiens aussi à remercier l'ensemble du corps enseignant et administratif de l'**Esprit School of Business** pour la qualité de la formation dispensée durant mon cursus en Licence Business Computing, parcours Business Intelligence.

Enfin, j'exprime ma reconnaissance à ma famille et à mes amis pour leur soutien constant et leurs encouragements.

Table des matières

Dédicaces	i
Remerciements	ii
Liste des acronymes	ix
Introduction générale	1
1 Cadre général du projet	2
1.1 Cadre du projet	2
1.2 Présentation de l'organisme d'accueil	3
1.2.1 Missions	4
1.2.2 Objectifs	4
1.3 Étude de l'existant	4
1.3.1 Description de la situation existante	4
1.3.2 Critique de la situation existante	5
1.3.3 Problématique	5
1.3.4 Solution proposée	5
1.3.5 Justification du choix de la solution	5
1.4 Méthodologie adoptée	6
1.4.1 Méthodes classiques et méthodes agiles	6
1.4.2 De Scrum à CRISP-DM	6
1.4.3 Choix et justification de CRISP-DM	7
1.4.4 Les phases de CRISP-DM appliquées au projet	7
1.4.5 Planification	7
2 Analyse et spécification des besoins	9
2.1 Identification des acteurs	9
2.2 Spécification des besoins	10
2.2.1 Besoins fonctionnels	10
2.2.2 Besoins non fonctionnels	10
2.3 Diagramme de cas d'utilisation global	11
2.4 Description textuelle des cas d'utilisation	11
2.4.1 Authentification et sécurité	11
2.4.2 Gestion du référentiel : agences et employés	13
2.4.3 Gestion commerciale : clients et objectifs	14
2.4.4 Pointage et workflows de demandes	14
2.4.5 Volet décisionnel	15
2.5 Environnement technique	17
3 Conception	19
3.1 Architecture du système	19
3.1.1 Architecture globale	19

3.1.2	Architecture physique	19
3.1.3	Architecture logique	20
3.2	Diagramme de classes	21
3.3	Modèle physique de données	22
3.4	Sécurité et contrôle d'accès	25
3.4.1	Authentification et session	25
3.4.2	Rôles et matrice des droits	25
3.5	Diagrammes de séquence	26
3.5.1	S'authentifier	26
3.5.2	Créer une agence	26
3.5.3	Gérer un client	27
3.5.4	Pointer un employé	28
3.5.5	Valider une demande	28
3.6	Cycle de vie d'une demande	28
3.6.1	Diagramme d'états-transitions	29
3.6.2	Diagramme d'activité	30
4	Volet décisionnel	31
4.1	Chaîne décisionnelle et processus ETL	31
4.2	Modèle en étoile de l'entrepôt	32
4.2.1	Comparaison des schémas de modélisation	32
4.2.2	Structure du modèle retenu	33
4.3	Mise en œuvre de la chaîne ETL en Python	35
4.3.1	Export des données sources	35
4.3.2	Détail des étapes de traitement	35
4.4	Tableau de bord et restitution Power BI	37
4.5	Intégration du modèle en étoile dans Power BI	38
4.5.1	Création des relations entre les tables	38
4.5.2	Exploitation dans les visualisations	38
4.6	Rapport décisionnel sous Power BI	40
4.6.1	Indicateurs et mesures (DAX)	40
4.6.2	Page de synthèse	40
4.6.3	Analyse des clients	42
4.6.4	Performance commerciale	43
4.6.5	Suivi de la présence	44
4.6.6	Navigation et interactivité	45
4.7	Segmentation des agences par apprentissage non supervisé	45
4.7.1	Données et variables	45
4.7.2	Prétraitement	46
4.7.3	Détermination du nombre de clusters	46
4.7.4	Comparaison des modèles de clustering	47
4.7.5	Résultats et interprétation	48
4.7.6	Apport décisionnel	49
5	Réalisation et déploiement	50
5.1	Environnement de travail	50
5.1.1	Environnement matériel	50
5.1.2	Outil de rédaction du rapport	50

5.1.3	Environnement logiciel	51
5.2	Interfaces de l'application	53
5.2.1	Authentification	53
5.2.2	Gestion des agences et des employés	53
5.2.3	Gestion des clients et des objectifs	55
5.2.4	Pointage des employés	56
5.3	Organisation du code	57
5.4	Déploiement de l'application	57
5.5	Tests et validation	58
	Conclusion générale et perspectives	59
	Bibliographie	60

Table des figures

1.1	Logo de la BTK – Banque Tuniso-Koweïtienne	3
1.2	Planning prévisionnel du projet (diagramme de Gantt selon CRISP-DM)	8
2.1	Diagramme de cas d'utilisation global	12
3.1	Architecture globale du système	20
3.2	Architecture physique (diagramme de déploiement 3-tiers)	21
3.3	Architecture logique en couches de l'application	22
3.4	Diagramme de classes du modèle métier	23
3.5	Schéma relationnel de la base de données	24
3.6	Diagramme de séquence – Authentification	26
3.7	Diagramme de séquence – Création d'une agence	27
3.8	Diagramme de séquence – Gestion d'un client	27
3.9	Diagramme de séquence – Pointage d'un employé	28
3.10	Diagramme de séquence – Validation d'une demande	29
3.11	Diagramme d'états-transitions – Cycle de vie d'une demande	29
3.12	Diagramme d'activité – Traitement d'une demande	30
4.1	Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution	32
4.2	Modèle décisionnel en étoile (vues V_BI_*)	34
4.3	Collecte : lecture des tables sources du réseau BTK (données réelles)	35
4.4	Datamart par agence sur les données réelles du réseau BTK (49 agences)	37
4.5	Contrôle de qualité : statistiques du datamart (données réelles du réseau BTK)	37
4.6	Tableau de bord décisionnel embarqué de l'application	38
4.7	Modèle de données Power BI : relations en étoile autour de la dimension AGENCE	39
4.8	Rapport Power BI — page « Résumé » (synthèse du réseau)	42
4.9	Rapport Power BI — page « Clients » (segmentation et territoire)	43
4.10	Rapport Power BI — page « Commercial » (objectifs et production)	43
4.11	Rapport Power BI — page « Présence » (assiduité et suivi RH)	44
4.12	Choix du nombre de clusters : méthode du coude et score de silhouette	46
4.13	Comparaison des modèles selon le score de silhouette	47
4.14	Segmentation des agences (K-Means) projetée par PCA	48
5.1	Logo de « $\text{\LaTeX}$ »	50
5.2	Logo de « Java (JDK 21) »	51
5.3	Logo de « Jakarta EE »	51
5.4	Logo de « Hibernate »	51
5.5	Logo d'« Oracle Database »	51
5.6	Logo de « WildFly / JBoss »	52
5.7	Logo d'« Apache Maven »	52
5.8	Logo d'« IntelliJ IDEA »	52
5.9	Logos de « Git » et « GitHub »	52
5.10	Logo d'« ApexCharts »	52
5.11	Logo de « Microsoft Power BI »	53

5.12	Logos de « Python », « pandas » et « scikit-learn »	53
5.13	Interface d'authentification	54
5.14	Interface de gestion des agences (sélection et consultation des membres) . . .	54
5.15	Interface de gestion des employés (ajout, filtres et liste)	55
5.16	Interface de gestion des clients (portefeuille BTK)	55
5.17	Suivi des objectifs commerciaux par agence et gestionnaire	56
5.18	Interface du module de pointage des employés	56

Liste des tableaux

1.1	Fiche signalétique de la BTK	3
1.2	Méthodes classiques vs méthodes agiles	6
1.3	Scrum vs CRISP-DM	7
1.4	Les six phases de CRISP-DM appliquées au projet	8
2.1	Identification des acteurs	9
2.2	Besoins non fonctionnels	10
2.3	Description textuelle du cas d'utilisation « S'authentifier »	11
2.4	Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »	13
2.5	Description textuelle du cas d'utilisation « Gérer les agences »	13
2.6	Description textuelle du cas d'utilisation « Gérer les employés »	14
2.7	Description textuelle du cas d'utilisation « Gérer les clients »	14
2.8	Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »	15
2.9	Description textuelle du cas d'utilisation « Pointer la présence »	15
2.10	Description textuelle du cas d'utilisation « Valider une demande »	16
2.11	Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »	16
2.12	Description textuelle du cas d'utilisation « Consulter le tableau de bord »	16
2.13	Description textuelle du cas d'utilisation « Segmenter les agences »	17
2.14	Environnement technique du projet	17
3.1	Description des principales tables de la base	24
3.2	Matrice des droits d'accès par rôle	25
4.1	Comparaison des schémas de modélisation des entrepôts de données	33
4.2	Table de faits et tables de dimensions du modèle en étoile	34
4.3	Indicateurs de performance (KPI) du volet décisionnel	34
4.4	Calcul des indicateurs lors de l'étape de transformation	36
4.5	Relations du modèle Power BI et leur cardinalité	39
4.6	Principales mesures DAX du rapport Power BI	41
4.7	Principaux KPI des tableaux de bord et leur intérêt décisionnel	41
4.8	Variables de segmentation (par agence)	46
4.9	Comparaison des modèles de clustering	47
4.10	Profil moyen des segments d'agences	48
4.11	Recommandations de pilotage par segment	49
5.1	Principaux modules du projet	57
5.2	Principaux scénarios de test fonctionnel	58

Liste des acronymes

Acronyme	Signification
BTK	Banque Tuniso-Koweïtienne
BI	Business Intelligence (informatique décisionnelle)
ETL	Extract, Transform, Load
IA	Intelligence Artificielle
ML	Machine Learning (apprentissage automatique)
ACP	Analyse en Composantes Principales (PCA)
ORM	Object-Relational Mapping
KPI	Key Performance Indicator (indicateur clé de performance)
PFE	Projet de Fin d'Études
UML	Unified Modeling Language
JPA	Jakarta Persistence API
EJB	Enterprise Java Beans
JSP	Jakarta Server Pages
JSTL	Jakarta Standard Tag Library
JTA	Jakarta Transaction API
JDBC	Java Database Connectivity
MVC	Modèle-Vue-Contrôleur
SGBD	Système de Gestion de Base de Données
WAR	Web Application Archive
CRUD	Create, Read, Update, Delete
HTTP	HyperText Transfer Protocol

Introduction générale

La transformation numérique du secteur bancaire impose aux établissements de disposer de systèmes d'information capables, d'une part, de centraliser la gestion de leur réseau d'agences et, d'autre part, d'exploiter leurs données pour piloter l'activité. La maîtrise de l'information — effectifs, portefeuille clients, objectifs commerciaux — constitue aujourd'hui un levier déterminant de performance et d'aide à la décision.

C'est dans ce contexte que s'inscrit ce projet de fin d'études, réalisé au sein de la **BTK – Banque Tuniso-Koweïtienne** en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'Esprit School of Business. Le projet consiste à concevoir et réaliser une application web de gestion des agences bancaires, adossée à un volet décisionnel d'aide à la décision.

L'application couvre la gestion des agences, des employés, des clients et des objectifs commerciaux, le **pointage (présence) des employés**, ainsi qu'un ensemble de circuits de validation (workflows) encadrant la création et la modification des données sensibles. À ces fonctionnalités transactionnelles s'ajoute un volet décisionnel : une chaîne **ETL** consolide les données dans un datamart qui alimente des rapports Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

Ce rapport, qui rend compte d'un projet conduit selon la méthodologie **CRISP-DM**, est organisé en cinq chapitres. Le **premier chapitre** présente le cadre général du projet : l'organisme d'accueil, l'étude de l'existant, la problématique et la méthodologie adoptée. Le **deuxième chapitre** est consacré à l'analyse et à la spécification des besoins (acteurs, besoins fonctionnels et non fonctionnels, cas d'utilisation). Le **troisième chapitre** détaille la conception : architecture, modèle de données et scénarios dynamiques. Le **quatrième chapitre** est dédié au **volet décisionnel** : la chaîne ETL, le modèle en étoile, les tableaux de bord et les rapports Power BI, ainsi que la segmentation des agences par apprentissage automatique. Le **cinquième et dernier chapitre** présente la **réalisation et le déploiement** de la solution. Une conclusion générale dresse le bilan du travail et ouvre des perspectives d'évolution.

Chapitre 1

Cadre général du projet

Introduction

Ce premier chapitre situe le projet dans son contexte. Il délimite d'abord le cadre du projet, présente ensuite l'organisme d'accueil et ses missions, puis analyse la situation existante afin d'en dégager la problématique et la solution proposée. Il se termine par la présentation de la **démarche** et de la **planification** du projet.

1.1 Cadre du projet

Le présent travail constitue un **projet de fin d'études** réalisé en vue de l'obtention de la Licence en Business Computing, parcours Business Intelligence, à l'**Esprit School of Business**. Il s'est déroulé dans le cadre d'un stage au sein de la **BTK – Banque Tuniso-Koweïtienne**, du **16 février au 16 juin 2026**. Le projet porte sur la conception et le développement d'une application web de gestion du réseau d'agences bancaires, adossée à un volet décisionnel d'aide à la décision.

Les objectifs assignés à ce projet sont les suivants :

- **centraliser** la gestion des agences, des employés, des clients et des objectifs commerciaux dans une base unique ;
- **sécuriser** l'accès par une gestion fine des rôles et des droits ;
- **formaliser** les opérations sensibles par des circuits de validation et en assurer la traçabilité ;
- **suivre la présence** des employés au moyen d'un module de pointage ;
- **restituer** des indicateurs d'aide à la décision via une chaîne ETL, un tableau de bord et des rapports Power BI ;

- **valoriser** les données par une segmentation des agences fondée sur l'apprentissage automatique.

1.2 Présentation de l'organisme d'accueil

La **Banque Tuniso-Koweïtienne** (BTK Bank) [1] est une banque universelle de droit tunisien, créée le 25 février 1981 dans le cadre d'une convention de coopération bilatérale entre la Tunisie et le Koweït. Constituée en société anonyme et régie par la loi bancaire n° 2016-48, elle dispose d'un capital social de 200 millions de dinars et a son siège social à Tunis. À travers un réseau d'agences couvrant l'ensemble du territoire tunisien, elle accompagne aussi bien les particuliers que les entreprises.



Figure 1.1 – Logo de la BTK – Banque Tuniso-Koweïtienne

Le tableau 1.1 en résume la fiche signalétique.

Table 1.1 – Fiche signalétique de la BTK

Élément	Renseignement
Dénomination sociale	Banque Tuniso-Koweïtienne (BTK Bank)
Forme juridique	Société Anonyme (S.A.)
Date de création	25 février 1981
Capital social	200 000 000 DT
Siège social	10 bis, Avenue Mohamed V, 1001 Tunis
Secteur d'activité	Banque universelle
Actionnaire de référence	Groupe Elloumi (60 %)
Réseau	5 directions régionales, 6 succursales, 45 agences
Effectif (2024)	≈ 400 collaborateurs

1.2.1 Missions

En tant que banque universelle, la BTK assure un ensemble de missions couvrant les principaux métiers bancaires :

- **Banque de détail** : comptes, cartes, crédits à la consommation et immobiliers, épargne et banque digitale destinés aux particuliers ;
- **Banque des entreprises** : financement de l'investissement et du fonctionnement, financement du commerce extérieur et gestion de trésorerie ;
- **Opérations internationales et de marché** : change, couverture du risque de change et placements ;
- **Innovation et digitalisation** : modernisation des services et offre en ligne.

1.2.2 Objectifs

À travers ces missions, la BTK poursuit plusieurs objectifs stratégiques :

- soutenir le tissu économique en finançant les entreprises et les projets d'investissement ;
- proposer des solutions financières adaptées aux ménages ;
- contribuer à la création d'emplois et à la dynamique de croissance nationale ;
- consolider sa solidité financière et accélérer sa transformation digitale.

1.3 Étude de l'existant

1.3.1 Description de la situation existante

La gestion actuelle du réseau repose en grande partie sur des traitements manuels et des fichiers bureautiques non centralisés. Les informations relatives aux agences, aux employés et aux clients sont saisies et conservées de manière hétérogène ; les demandes de création ou de modification circulent par des canaux informels ; et la présence des employés n'est suivie par aucun dispositif formalisé.

1.3.2 Critique de la situation existante

Cette organisation présente plusieurs limites :

- dispersion et redondance des données, sources d'incohérences ;
- absence de circuit de validation formalisé pour les opérations sensibles ;
- absence de suivi de présence (pointage) des employés ;
- faible traçabilité des actions et des décisions ;
- difficulté à produire des indicateurs consolidés pour le pilotage ;
- gestion des droits d'accès insuffisamment maîtrisée.

1.3.3 Problématique

La principale problématique qui se pose est la suivante : comment **centraliser et fiabiliser** la gestion du réseau d'agences bancaires, **encadrer** les opérations sensibles par des circuits de validation, **assurer le suivi de présence** des employés, et **exploiter** les données ainsi structurées pour fournir aux responsables des indicateurs d'aide à la décision ?

1.3.4 Solution proposée

La solution retenue consiste en une **application web centralisée**, structurée autour d'une base de données Oracle, qui couvre la gestion des agences, des employés, des clients et des objectifs, gère les rôles et la sécurité des accès, assure le **pointage** de présence des employés, et formalise les demandes de création et de modification au moyen de **workflows de validation**. Un **volet décisionnel** complète le dispositif : une chaîne ETL consolide les données dans un datamart qui alimente à la fois Microsoft Power BI, un tableau de bord embarqué et une **segmentation des agences** par apprentissage automatique.

1.3.5 Justification du choix de la solution

Le choix d'une application web centralisée se justifie par plusieurs facteurs : l'**accessibilité** (un simple navigateur suffit, sans installation sur les postes), la **centralisation** des données dans une base unique garantissant leur cohérence, la **sécurité** offerte par une gestion fine des rôles et

des workflows de validation, et l'**ouverture décisionnelle** permise par l'exposition des données à des outils d'analyse et de *reporting*.

1.4 Méthodologie adoptée

Le choix d'une méthodologie adaptée conditionne le bon déroulement du projet. Ce projet étant **centré sur les données** — consolidation, tableaux de bord et apprentissage automatique —, la méthodologie **CRISP-DM** a été retenue, après comparaison des approches classiques, agiles et décisionnelles.

1.4.1 Méthodes classiques et méthodes agiles

Les **méthodes classiques** (cycle en cascade, cycle en V) reposent sur un enchaînement **séquentiel** de phases planifiées dès le départ ; elles conviennent aux besoins stables mais s'adaptent mal au changement. Les **méthodes agiles** privilégient au contraire une approche **itérative et incrémentale**, fondée sur la collaboration continue et la livraison fréquente d'incrément [2]. Le tableau 1.2 oppose les deux familles.

Table 1.2 – Méthodes classiques vs méthodes agiles

Critère	Méthodes classiques	Méthodes agiles
Approche	Séquentielle et planifiée (cascade, cycle en V)	Itérative et incrémentale
Flexibilité	Faible : les changements sont coûteux	Élevée : adaptation continue
Livraison	En fin de projet	Par incréments fréquents
Implication du client	Au début et à la fin	Tout au long du projet
Détection des risques	Tardive	Précoce

1.4.2 De Scrum à CRISP-DM

Scrum [2] est un cadre agile de *développement logiciel* : il organise le travail en *sprints*, avec des rôles et des cérémonies, et vise la livraison d'incrément fonctionnels. **CRISP-DM** [3] (*Cross-Industry Standard Process for Data Mining*) est en revanche une méthodologie dédiée

aux *projets de données et décisionnels* : elle structure le travail en **six phases** orientées vers la compréhension, la préparation et l'exploitation des données. Le tableau 1.3 les compare.

Table 1.3 – Scrum vs CRISP-DM

Critère	Scrum	CRISP-DM
Domaine	Développement logiciel	Projets de données / décisionnel
Structure	Sprints, rôles, cérémonies	Six phases orientées données
Centré sur	Fonctionnalités livrables	Compréhension et exploitation des données
Cycle	Itératif (sprints)	Itératif (retours entre phases)
Livrable final	Incrément logiciel	Solution décisionnelle déployée
Adapté à	Applications et produits	Analyse de données, BI, apprentissage automatique

1.4.3 Choix et justification de CRISP-DM

Le cœur de ce projet est un **système décisionnel** : consolidation des données par une chaîne ETL, restitution par des tableaux de bord et Power BI, et segmentation des agences par apprentissage automatique. Pour ce type de projet **orienté données**, **CRISP-DM** constitue la méthodologie de référence : ses phases épousent naturellement le déroulement du travail, de la compréhension du besoin métier jusqu'au déploiement de la solution. Scrum, centré sur la seule production logicielle, ne couvre pas les étapes propres à l'analyse et à la valorisation des données ; **CRISP-DM a donc été retenu**, tout en intégrant des pratiques agiles (itérations et validations régulières avec l'encadrement) au sein de ses phases.

1.4.4 Les phases de CRISP-DM appliquées au projet

CRISP-DM se décompose en six phases [3], susceptibles d'être rejouées en cycles. Le tableau 1.4 en précise le contenu dans le cadre du projet et la correspondance avec les chapitres de ce rapport.

1.4.5 Planification

Le projet s'est déroulé sur quatre mois, du 16 février au 16 juin 2026. La figure 1.2 en présente le planning prévisionnel sous forme de diagramme de Gantt, organisé selon les phases

Table 1.4 – Les six phases de CRISP-DM appliquées au projet

Phase	Contenu dans le projet	Chapitre
Compréhension métier	Cadre du projet, étude de l'existant, besoins et objectifs.	Ch. 1-2
Compréhension des données	Sources opérationnelles, entités et modèle de données.	Ch. 2-3
Préparation des données	Chaîne ETL : nettoyage, agrégation, datamart en étoile.	Ch. 4
Modélisation	Indicateurs, tableaux de bord et segmentation (K-Means).	Ch. 4
Évaluation	Comparaison des modèles et validation des résultats.	Ch. 4
Déploiement	Réalisation, restitution Power BI et déploiement de la solution.	Ch. 5

de CRISP-DM.

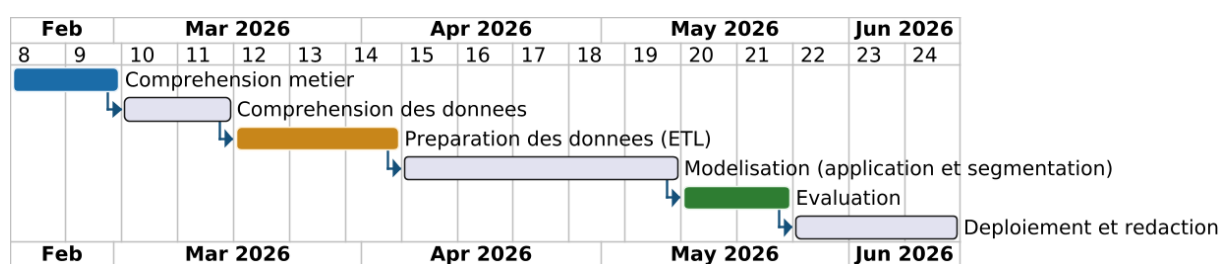


Figure 1.2 – Planning prévisionnel du projet (diagramme de Gantt selon CRISP-DM)

Conclusion

Ce chapitre a posé le cadre du projet : l'organisme d'accueil et ses missions, l'étude de l'existant qui a mis en évidence la problématique, la solution proposée, ainsi que la démarche et la planification retenues. Le chapitre suivant est consacré à **l'analyse et à la spécification des besoins**.

Chapitre 2

Analyse et spécification des besoins

Introduction

Ce chapitre est consacré à l'**analyse** du système à réaliser. Il identifie d'abord les acteurs qui interagissent avec l'application, spécifie ensuite les besoins fonctionnels et non fonctionnels, présente le diagramme de cas d'utilisation global, puis détaille par une description textuelle les principaux cas d'utilisation, regroupés par module fonctionnel.

2.1 Identification des acteurs

Trois profils interagissent avec l'application, conformément aux rôles gérés par le module de sécurité (table SECURITY_USERS). Le tableau 2.1 en précise les responsabilités.

Table 2.1 – Identification des acteurs

Acteur	Responsabilités
Administrateur (ADMIN)	Gère les agences, les employés et les clients ; valide les demandes de création de clients et d'employés ; consulte le journal d'audit ; supervise le volet décisionnel.
Directeur commercial (DIRECTEUR_COMMERCIAL)	Soumet les demandes de création (clients, employés) ; valide les demandes de modification des données des employés ; consulte les objectifs et les indicateurs.
Utilisateur (USER)	Consulte les données autorisées, effectue son pointage et soumet des demandes de modification de ses propres informations.

2.2 Spécification des besoins

2.2.1 Besoins fonctionnels

Le système doit permettre :

- l'authentification sécurisée et la gestion des rôles et des droits d'accès (ADMIN, DIRECTEUR_COMMERCIAL, USER);
- la gestion (CRUD) des agences, des employés et des clients;
- la consultation des objectifs commerciaux et le suivi de la production;
- le **pointage (présence) des employés** : saisie de l'heure d'arrivée et de départ, statut (présent / retard / absent), en mode automatique (bouton) ou manuel (administrateur);
- la soumission et la validation des demandes (clients, employés, modifications) selon le cycle EN_ATTENTE → EN_COURS → EFFECTUE/REFUSE;
- la notification des demandes en attente et la journalisation des actions;
- la consolidation des données par une chaîne **ETL** et la restitution d'indicateurs via un tableau de bord et Power BI;
- la **segmentation des agences** par apprentissage non supervisé.

2.2.2 Besoins non fonctionnels

Le tableau 2.2 récapitule les besoins non fonctionnels.

Table 2.2 – Besoins non fonctionnels

Besoin	Description
Sécurité	Authentification, gestion des rôles et protection des opérations sensibles par des workflows de validation.
Fiabilité	Intégrité des données garantie par le SGBD et les transactions JTA.
Performance	Temps de réponse acceptable sur les volumes manipulés.
Ergonomie	Interface claire et cohérente, navigation par menu latéral.
Maintenabilité	Architecture en couches, code modulaire et faiblement couplé.
Portabilité	Application Jakarta EE standard, déployable sur serveur compatible.

2.3 Diagramme de cas d'utilisation global

La figure 2.1 présente le **diagramme de cas d'utilisation global**, formalisé selon la notation **UML** [4], qui représente d'un point de vue fonctionnel les interactions entre les acteurs et les services offerts par le système.

On y distingue les trois acteurs et leurs cas d'utilisation respectifs. Le cas *S'authentifier* est commun à tous ; il est relié aux autres par une relation «include», l'accès aux fonctionnalités exigeant une authentification préalable. L'**administrateur** concentre les cas de gestion et de validation ; le **directeur commercial** soumet les demandes et consulte les indicateurs ; l'**utilisateur** se limite à la consultation, au pointage et aux demandes le concernant.

2.4 Description textuelle des cas d'utilisation

Cette section détaille, sous forme de fiches, les principaux cas d'utilisation, regroupés par module fonctionnel.

2.4.1 Authentification et sécurité

Ce module regroupe les cas d'utilisation qui conditionnent l'accès à l'application. Le premier, *S'authentifier*, en contrôle l'entrée : le tableau 2.3 en détaille le déroulement.

Table 2.3 – Description textuelle du cas d'utilisation « S'authentifier »

Titre	S'authentifier
Objectif	Permettre à l'utilisateur de se connecter à l'application avec son identifiant et son mot de passe.
Acteur	Administrateur / Directeur commercial / Utilisateur
Pré-condition	L'utilisateur doit disposer d'un compte actif dans la table SECURITY_USERS.
Post-condition	Une session est ouverte et l'utilisateur est redirigé vers le tableau de bord correspondant à son rôle.
Scénario nominal	(1) L'utilisateur saisit son identifiant et son mot de passe ; (2) le SecurityService vérifie les informations sur la table de sécurité ; (3) en cas de succès, une session est ouverte et l'utilisateur est redirigé ; (4) sinon, un message d'erreur est affiché.

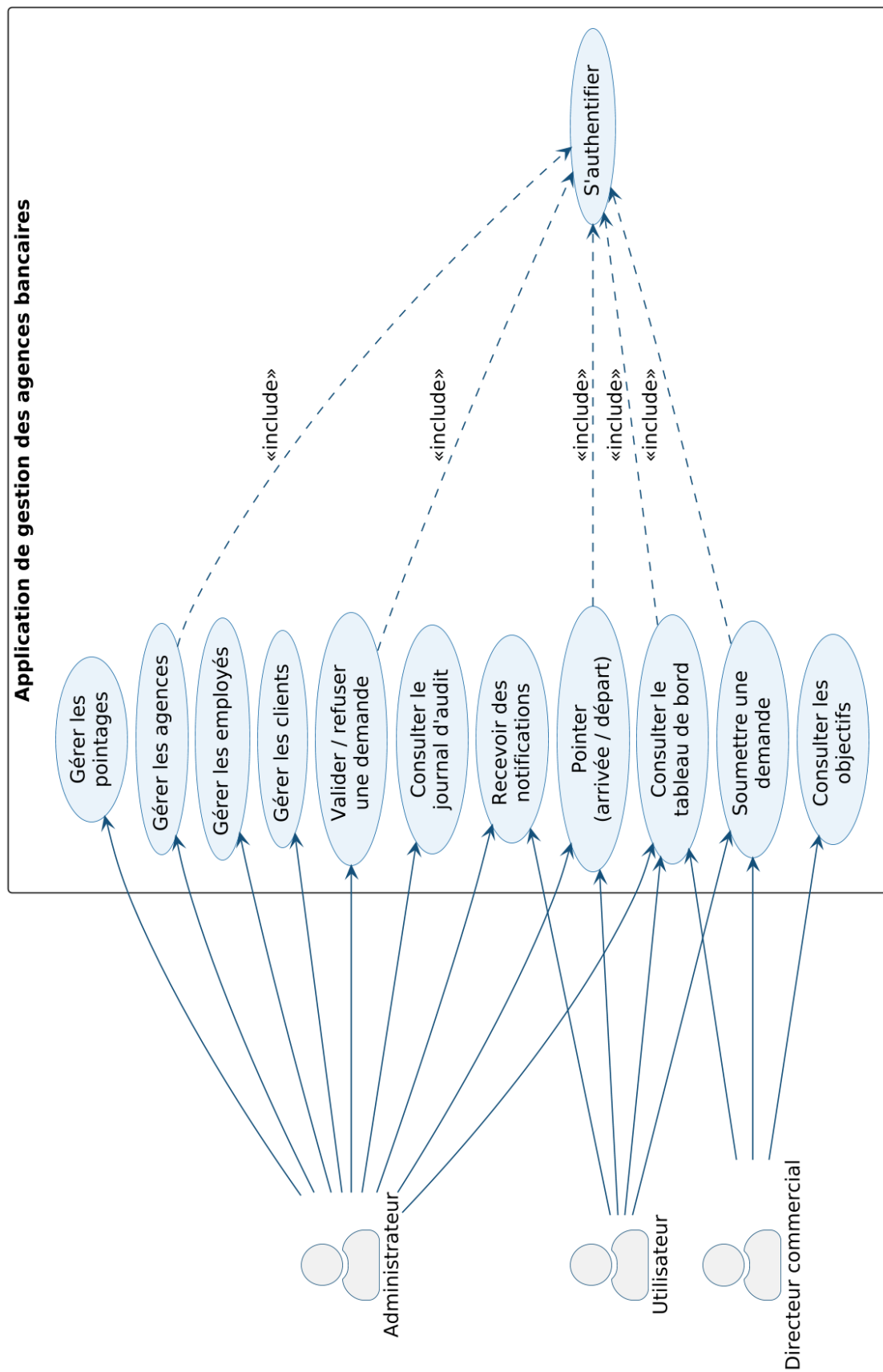


Figure 2.1 – Diagramme de cas d'utilisation global

Une fois connecté, l'administrateur attribue à chaque utilisateur un rôle qui détermine ses droits d'accès, comme le précise le tableau 2.4.

Table 2.4 – Description textuelle du cas d'utilisation « Gérer les rôles et les droits d'accès »

Titre	Gérer les rôles et les droits d'accès
Objectif	Permettre à l'administrateur d'attribuer un rôle à chaque utilisateur afin de contrôler l'accès aux fonctionnalités.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	Le rôle est enregistré et les droits d'accès de l'utilisateur sont mis à jour.
Scénario nominal	(1) L'administrateur consulte la liste des utilisateurs ; (2) il sélectionne un utilisateur ; (3) il lui attribue un rôle (ADMIN, DIRECTEUR_COMMERCIAL ou USER) ; (4) le système applique le contrôle d'accès correspondant.

2.4.2 Gestion du référentiel : agences et employés

Ce module couvre la gestion du référentiel structurant du réseau. Le premier cas, *Gérer les agences* (tableau 2.5), administre les agences.

Table 2.5 – Description textuelle du cas d'utilisation « Gérer les agences »

Titre	Gérer les agences
Objectif	Permettre à l'administrateur de créer, consulter, modifier et supprimer les agences du réseau.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié avec le rôle ADMIN.
Post-condition	L'agence créée ou modifiée est enregistrée dans la base et la liste est mise à jour.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des agences ; (2) il sélectionne une action (créer, modifier, supprimer) ; (3) il saisit ou met à jour les informations de l'agence ; (4) l'AgenceService persiste la modification et la liste est rafraîchie.

Chaque agence étant animée par des employés, le second cas, *Gérer les employés* (tableau 2.6), les rattache à leur agence.

Table 2.6 – Description textuelle du cas d'utilisation « Gérer les employés »

Titre	Gérer les employés
Objectif	Permettre à l'administrateur de gérer les employés et de les rattacher à leur agence.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et l'agence de rattachement doit exister.
Post-condition	L'employé est enregistré et rattaché à son agence.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des employés ; (2) il saisit les informations de l'employé ; (3) il sélectionne l'agence de rattachement ; (4) l'UtilisateurService enregistre l'employé associé à l'agence.

2.4.3 Gestion commerciale : clients et objectifs

Ce module concerne la gestion commerciale du réseau : le portefeuille clients et le suivi des objectifs de production. Le premier cas, *Gérer les clients* (tableau 2.7), administre le portefeuille.

Table 2.7 – Description textuelle du cas d'utilisation « Gérer les clients »

Titre	Gérer les clients
Objectif	Permettre à l'administrateur de gérer le portefeuille clients et d'affecter chaque client à un gestionnaire.
Acteur	Administrateur
Pré-condition	L'administrateur doit être authentifié et le gestionnaire de rattachement doit exister.
Post-condition	Le client est enregistré et associé à son gestionnaire dans la base.
Scénario nominal	(1) L'administrateur accède à l'interface de gestion des clients ; (2) il saisit ou met à jour les informations du client ; (3) il sélectionne le gestionnaire responsable ; (4) le ClientService enregistre le client et met à jour le portefeuille.

Ce portefeuille alimente le suivi de la performance : le second cas, *Suivre les objectifs commerciaux* (tableau 2.8), en restitue la production par agence et par période.

2.4.4 Pointage et workflows de demandes

Ce module traite du suivi de la présence des employés et du circuit de validation des opérations sensibles. Le premier cas, *Pointer la présence* (tableau 2.9), enregistre les arrivées et

Table 2.8 – Description textuelle du cas d'utilisation « Suivre les objectifs commerciaux »

Titre	Suivre les objectifs commerciaux
Objectif	Permettre au directeur commercial de consulter les objectifs et la production par agence.
Acteur	Directeur commercial / Administrateur
Pré-condition	L'acteur doit être authentifié et des objectifs doivent être renseignés pour la période.
Post-condition	Les indicateurs de production (comptes, crédits, épargne) sont restitués par agence.
Scénario nominal	(1) L'acteur accède à l'interface des objectifs ; (2) il sélectionne une agence et une période ; (3) l'ObjectifService agrège la production ; (4) les indicateurs sont affichés.

départs avec détermination automatique du statut.

Table 2.9 – Description textuelle du cas d'utilisation « Pointer la présence »

Titre	Pointer la présence
Objectif	Permettre à l'employé d'enregistrer son arrivée et son départ, avec détermination automatique du statut.
Acteur	Utilisateur (employé)
Pré-condition	L'employé doit être authentifié.
Post-condition	Le pointage du jour est enregistré avec son statut (PRESENT ou RETARD).
Scénario nominal	(1) L'employé clique sur « Pointer » ; (2) le PointageService relève l'heure courante et recherche le pointage du jour ; (3) s'il n'existe pas, il détermine le statut (RETARD au-delà de 9 h) et l'enregistre ; (4) sinon, il met à jour l'enregistrement existant.

Au-delà du pointage, les opérations sensibles suivent un circuit de validation : le second cas, *Valider une demande* (tableau 2.10), en formalise l'approbation ou le refus.

2.4.5 Volet décisionnel

Ce dernier module rassemble les cas d'utilisation du volet décisionnel, de la préparation des données jusqu'à leur exploitation analytique. Le premier, *Alimenter le datamart* (tableau 2.11), consolide les données opérationnelles par la chaîne ETL.

Une fois le datamart alimenté, il est restitué aux responsables : le deuxième cas, *Consulter le tableau de bord* (tableau 2.12), en offre une vue synthétique.

Table 2.10 – Description textuelle du cas d'utilisation « Valider une demande »

Titre	Valider une demande
Objectif	Permettre à l'administrateur d'approuver ou de refuser une demande soumise.
Acteur	Administrateur
Pré-condition	Une demande au statut EN_ATTENTE doit exister.
Post-condition	Le statut de la demande évolue vers EFFECTUE ou REFUSE et l'action est journalisée.
Scénario nominal	(1) L'administrateur consulte les demandes en attente ; (2) il sélectionne une demande ; (3) il l'approuve ou la refuse avec un commentaire ; (4) le service met à jour le statut, applique la modification le cas échéant et journalise l'action.

Table 2.11 – Description textuelle du cas d'utilisation « Alimenter le datamart (ETL) »

Titre	Alimenter le datamart (ETL)
Objectif	Consolider les données opérationnelles en un datamart agrégé par agence.
Acteur	Processus ETL / Administrateur
Pré-condition	Les données sources (agences, employés, clients, objectifs, pointage) doivent être disponibles.
Post-condition	Le datamart en étoile est produit et mis à la disposition de la restitution et de la segmentation.
Scénario nominal	(1) L'étape <i>Extract</i> lit les données sources ; (2) l'étape <i>Transform</i> les nettoie et les agrège par agence ; (3) l'étape <i>Load</i> écrit le datamart (dim_agence, fait_agence).

Table 2.12 – Description textuelle du cas d'utilisation « Consulter le tableau de bord »

Titre	Consulter le tableau de bord
Objectif	Restituer aux responsables une vision synthétique de l'activité.
Acteur	Administrateur / Directeur commercial
Pré-condition	L'acteur doit être authentifié.
Post-condition	Les indicateurs clés (effectifs, présence, production) sont affichés.
Scénario nominal	(1) L'acteur se connecte ; (2) le <code>HomeServlet</code> calcule les statistiques ; (3) la page <code>home.jsp</code> affiche les cartes d'indicateurs et les graphiques ApexCharts.

Enfin, le troisième cas, *Segmenter les agences* (tableau 2.13), exploite ce même datamart pour regrouper les agences en profils homogènes par apprentissage non supervisé.

Table 2.13 – Description textuelle du cas d'utilisation « Segmenter les agences »

Titre	Segmenter les agences
Objectif	Regrouper automatiquement les agences en profils homogènes à partir de leurs indicateurs d'activité.
Acteur	Analyste décisionnel / Administrateur
Pré-condition	Le datamart agrégé par agence (<i>fait_agence</i>) doit être disponible.
Post-condition	Les agences sont réparties en segments cohérents, caractérisés et exploitables pour la décision.
Scénario nominal	(1) L'analyste lance la segmentation ; (2) le script standardise les variables ; (3) il détermine le nombre de clusters puis compare les modèles ; (4) il retient le meilleur modèle, projette en PCA et restitue le profil de chaque segment.

2.5 Environnement technique

La réalisation de la solution s'appuie sur un ensemble cohérent de technologies, réparties entre le développement de l'application avec **Jakarta EE** [5], la base de données **Oracle** [6] et le volet décisionnel sous **Power BI** [7]. Le tableau 2.14 en récapitule les principales et précise le rôle de chacune.

Table 2.14 – Environnement technique du projet

Catégorie	Technologie	Rôle
Langage	Java (JDK 21)	Langage principal de l'application.
Plateforme	Jakarta EE 10	Servlets, JSP, EJB et JPA : structuration en couches.
Persistence	Hibernate (JPA)	Mapping objet-relationnel entre entités et base.
Base de données	Oracle Database	Données opérationnelles et vues décisionnelles.
Serveur	WildFly / JBoss	Serveur d'applications (déploiement du WAR).
Build	Apache Maven	Gestion des dépendances et production du livrable.
Visualisation Web	ApexCharts	Graphiques du tableau de bord embarqué.
Décisionnel	Microsoft Power BI	Rapports analytiques connectés à Oracle.
ETL et IA	Python (pandas, scikit-learn)	Chaîne ETL et segmentation des agences.
Versionnement	Git / GitHub	Gestion de versions et hébergement du code.

Ce socle technique, retenu pour sa robustesse et sa large diffusion, est mis en œuvre tout

au long du projet ; l'environnement de travail et les outils correspondants sont présentés plus en détail au moment de la réalisation.

Conclusion

Ce chapitre a identifié les acteurs, spécifié les besoins fonctionnels et non fonctionnels, formalisé les cas d'utilisation et présenté l'environnement technique du système. Le chapitre suivant traduit cette analyse en une **conception** détaillée : architecture, modèle de classes et scénarios dynamiques.

Chapitre 3

Conception

Introduction

Ce chapitre traduit l'analyse en une **conception** détaillée du système. Il présente d'abord l'architecture (globale, physique et logique), puis le modèle statique du domaine (diagramme de classes) et enfin les scénarios dynamiques des fonctionnalités principales (diagrammes de séquence, d'états-transitions et d'activité).

3.1 Architecture du système

L'architecture définit l'organisation des composants du système et la manière dont ils communiquent. Elle est présentée à trois niveaux : globale, physique et logique.

3.1.1 Architecture globale

L'architecture globale (figure 3.1) offre une vue d'ensemble des composants. Un poste client (navigateur) dialogue en HTTP/HTTPS avec l'application web déployée sur **WildFly**, structurée en couches (présentation, contrôle, métier, persistance) ; celle-ci accède à la base **Oracle** en JDBC. La base alimente, d'une part, **Power BI** via les vues V_BI_* et, d'autre part, la **chaîne décisionnelle Python** (ETL puis segmentation).

3.1.2 Architecture physique

L'architecture physique (figure 3.2) représente le **déploiement** des composants sur les nœuds matériels selon une organisation **3-tiers** : le poste client, le serveur d'applications WildFly

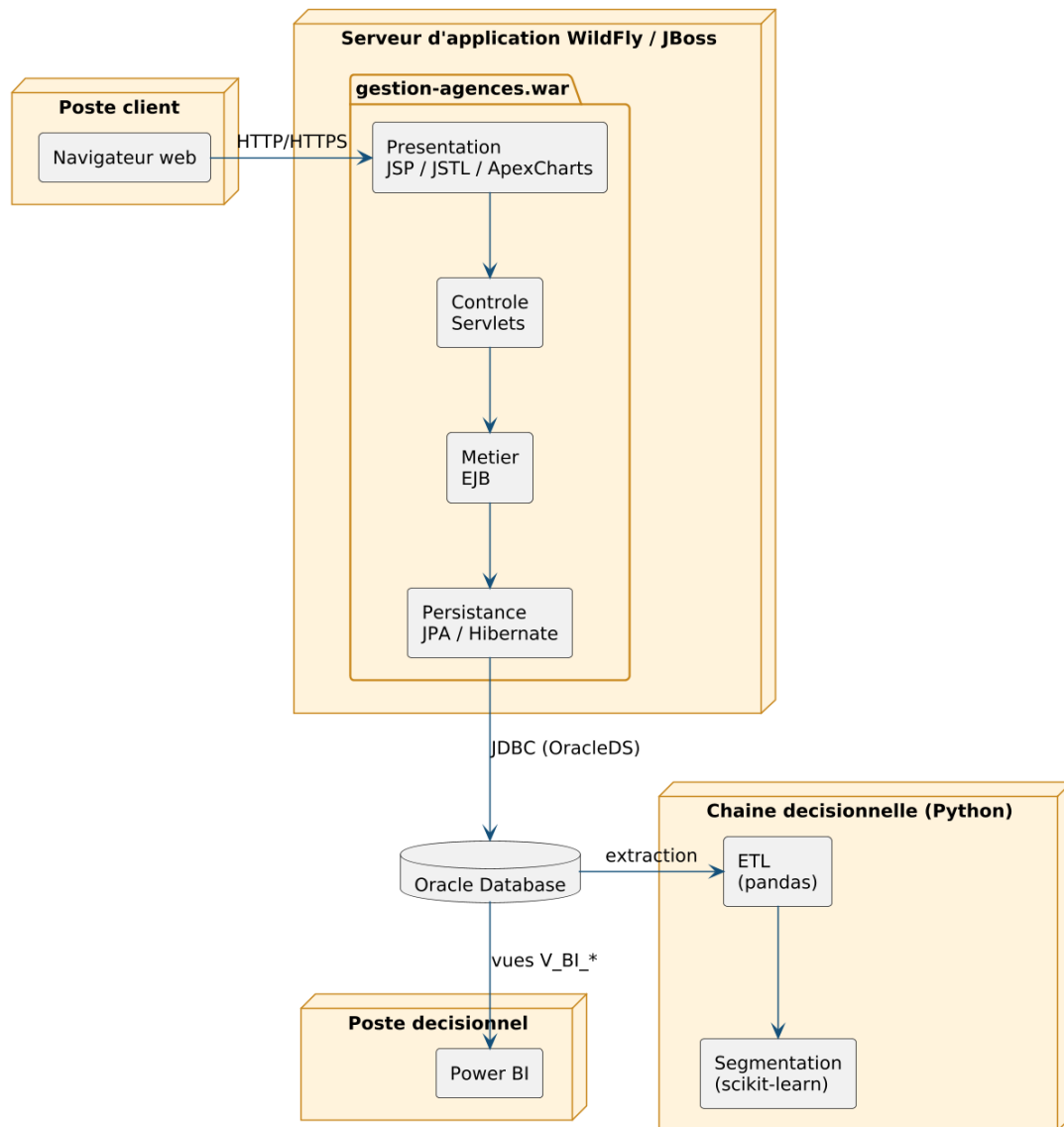


Figure 3.1 – Architecture globale du système

hébergeant `gestion-agences.war` (accès à la base via la source de données JTA OracleDS), le serveur de base de données Oracle (FREEPDB1) et le poste décisionnel exécutant Power BI.

3.1.3 Architecture logique

L'architecture logique (figure 3.3) décompose le système en **couches** faiblement couplées suivant le patron **MVC**, conformément aux pratiques de la plateforme Jakarta EE [8] : la couche *présentation* (JSP/JSTL, ApexCharts), la couche *contrôle* (Servlets et le filtre `NotificationFilter`), la couche *métier* (services EJB) et la couche *persistance* (entités JPA et Hibernate [9] dialoguant avec Oracle via l'`EntityManager`).

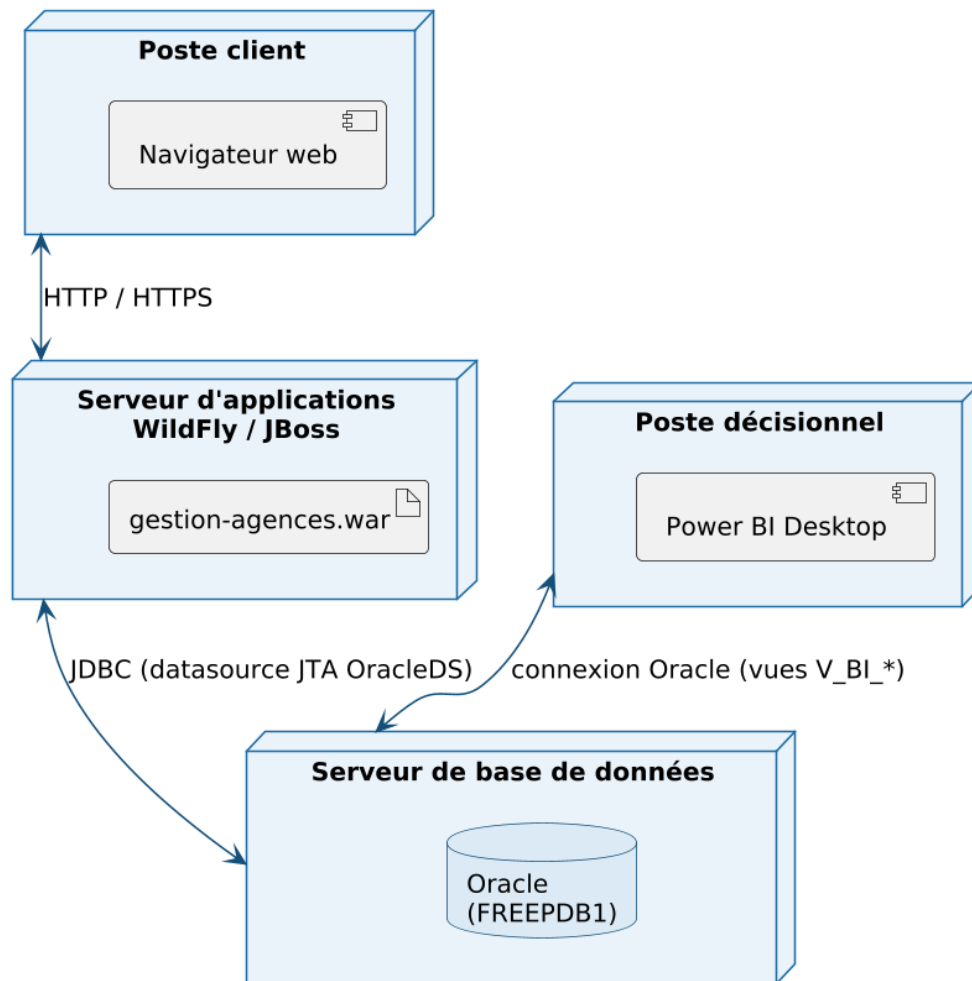


Figure 3.2 – Architecture physique (diagramme de déploiement 3-tiers)

3.2 Diagramme de classes

La figure 3.4 présente le **diagramme de classes** du modèle métier [10], qui représente les entités persistantes du domaine, leurs attributs et les associations qui les relient.

L'entité *Utilisateur* occupe une position centrale. Le rattachement d'un *Utilisateur*, d'un *Client* ou d'un *Objectif* à son *Agence*, ainsi que d'un *Client* à son *Gestionnaire*, est matérialisé dans le code par des **clés étrangères** (SK_AGENCE, SK_GESTIONNAIRE). Les entités de demande (*DemandeClient*, *DemandeEmploye*, *DemandeModification*, *DemandeAgence*), ainsi que *JournalAction*, *Notification* et *Pointage*, référencent un *Utilisateur* par une association JPA @ManyToMany; *DemandeAgence* possède en outre une association @ManyToMany avec *Utilisateur* (utilisateursDemandes). Enfin, *SecurityUser* est relié à *Utilisateur* par une dépendance d'authentification. Le modèle ne comporte ni héritage ni composition :

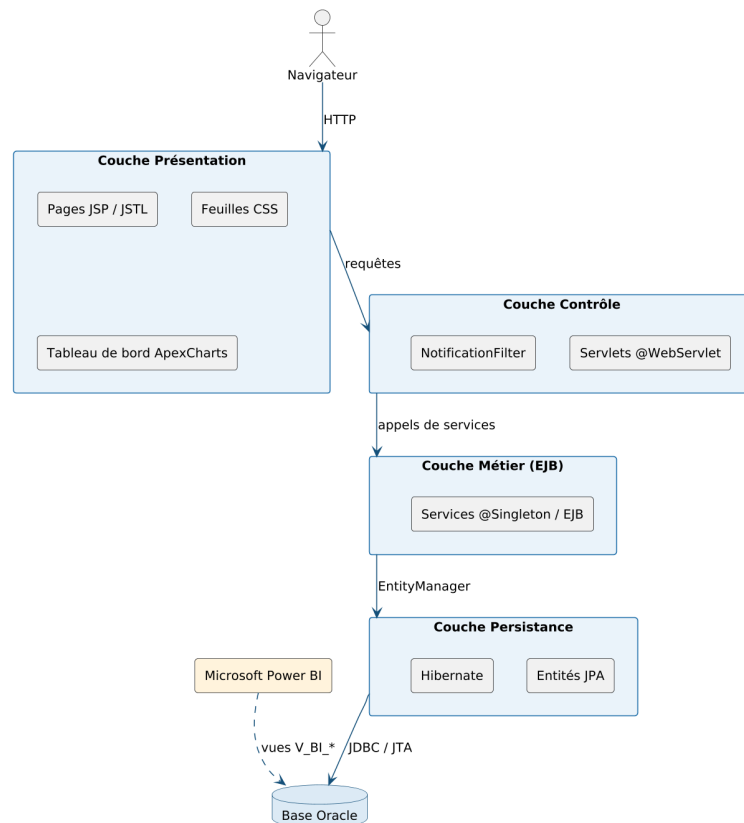


Figure 3.3 – Architecture logique en couches de l'application

les liens sont des associations, dont certaines sont implémentées par des clés étrangères et d'autres par des relations JPA explicites.

3.3 Modèle physique de données

Au modèle de classes correspond, au niveau de la base de données, un **schéma relationnel** : chaque entité persistante est traduite en une table Oracle [6], chaque attribut en une colonne et chaque association en une **clé étrangère**. La figure 3.5 présente ce schéma relationnel, obtenu par *mapping* objet-relationnel (JPA/Hibernate) à partir des entités du domaine.

La table AGENCE constitue le référentiel central autour duquel gravitent les employés (B_UTILISATEURS), les clients (B_CLIENTS) et les objectifs (B_OBJECTIF), reliés par la clé SK_AGENCE. La table B_UTILISATEURS est à son tour référencée par les pointages, les demandes et le journal d'audit. Le tableau 3.1 décrit le rôle des principales tables du schéma.

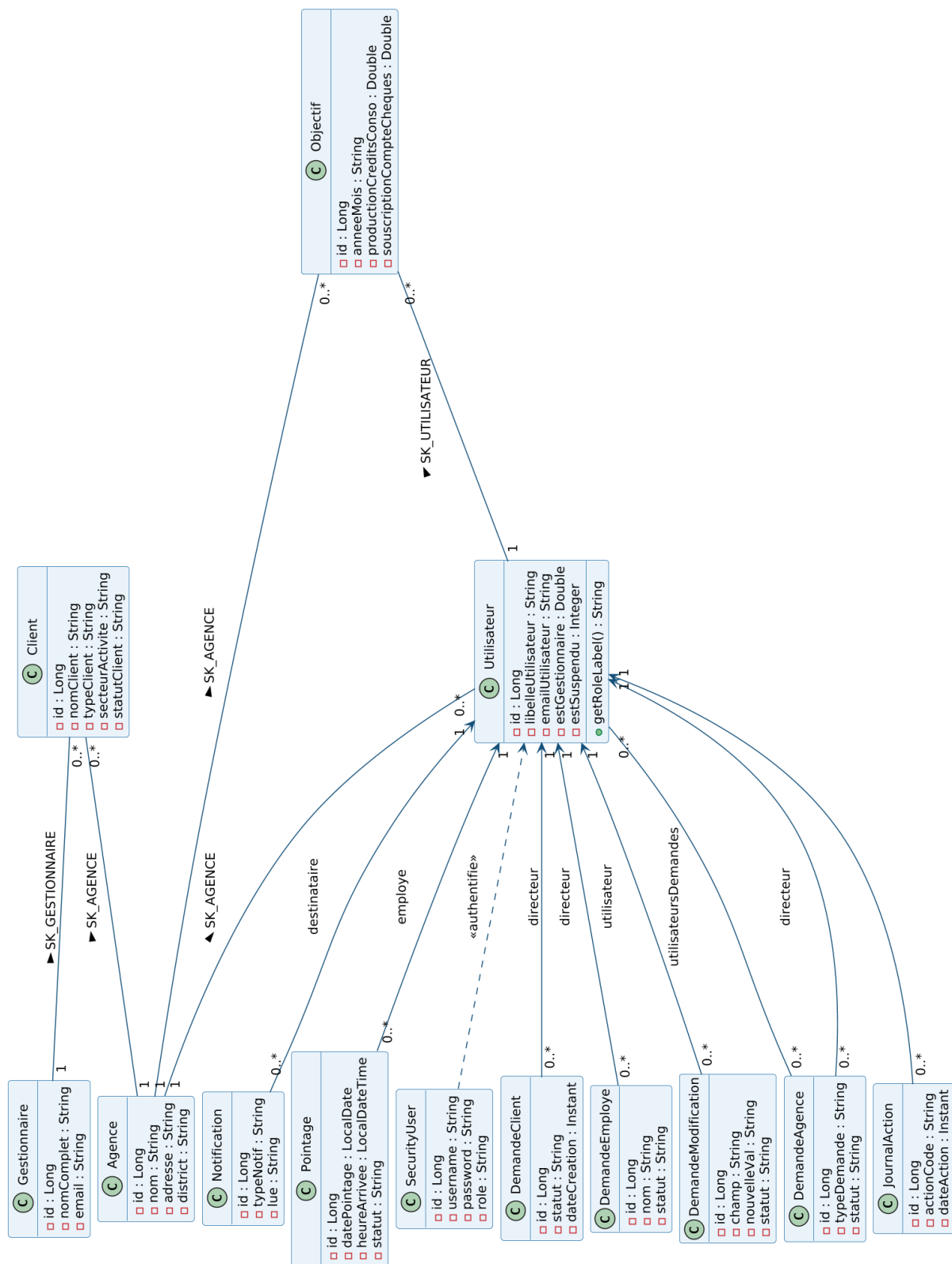


Figure 3.4 – Diagramme de classes du modèle métier

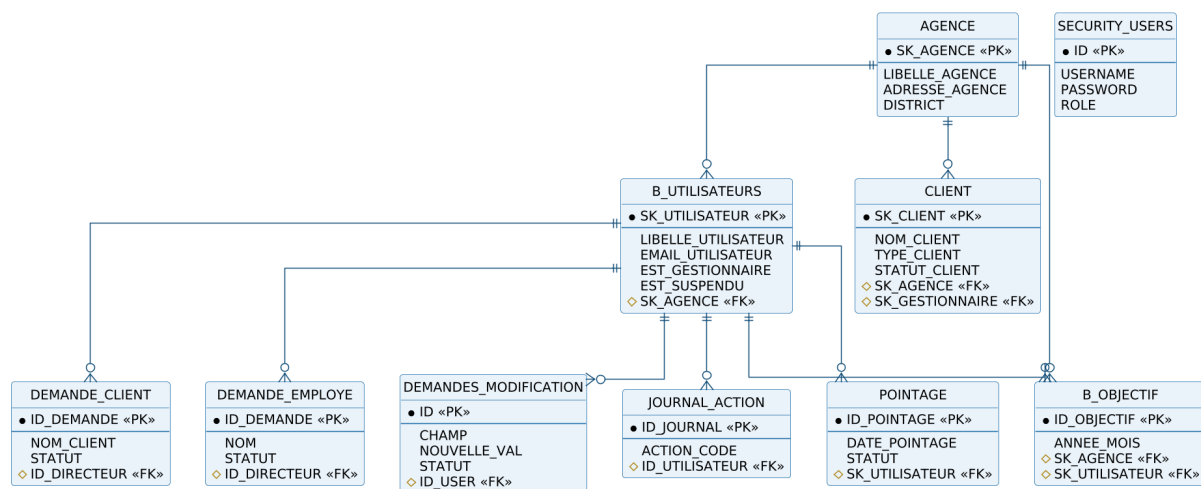


Figure 3.5 – Schéma relationnel de la base de données

Table 3.1 – Description des principales tables de la base

Table	Rôle	Clé primaire
AGENCE	Référentiel des agences du réseau (libellé, adresse, district).	SK_AGENCE
B_UTILISATEURS	Employés et utilisateurs de l'application.	SK_UTILISATEUR
B_CLIENTS	Portefeuille clients (type, statut, secteur, adresse).	SK_CLIENT
B_OBJECTIF	Objectifs commerciaux et production par période.	ID_OBJECTIF
POINTAGE	Pointages de présence des employés.	ID_POINTAGE
GESTIONNAIRE	Gestionnaires et rattachement de portefeuille.	SK_UTILISATEUR
SECURITY_USERS	Comptes, mots de passe et rôles (sécurité).	ID
DEMANDE_CLIENT	Demandes de création de clients.	ID_DEMANDE
DEMANDE_EMPLOYE	Demandes de création d'employés.	ID_DEMANDE
DEMANDES_MODIF.	Demandes de modification de données.	ID
JOURNAL_ACTION	Journal d'audit des actions.	ID_JOURNAL
NOTIFICATION	Notifications adressées aux utilisateurs.	ID

3.4 Sécurité et contrôle d'accès

La sécurité de l'application repose sur trois mécanismes complémentaires : l'authentification, la gestion des rôles et le filtrage des requêtes.

3.4.1 Authentification et session

Les comptes sont stockés dans la table `SECURITY_USERS` (identifiant, mot de passe et rôle). Le `LoginServlet` délègue la vérification au `SecurityService`; en cas de succès, une `HttpSession` est ouverte et l'Utilisateur métier correspondant y est associé. Le filtre `NotificationFilter` intercepte ensuite les requêtes pour vérifier la présence d'une session valide avant tout accès aux pages protégées, et redirige vers la page de connexion dans le cas contraire.

3.4.2 Rôles et matrice des droits

Trois rôles structurent les droits d'accès. Le tableau 3.2 présente la matrice des permissions par fonctionnalité.

Table 3.2 – Matrice des droits d'accès par rôle

Fonctionnalité	ADMIN	DIR. COMM.	USER
Gestion des agences, employés, clients	Oui	Non	Non
Validation des demandes	Oui	Modif. employés	Non
Soumission de demandes	Oui	Oui	Ses informations
Consultation des objectifs et indicateurs	Oui	Oui	Autorisé
Pointage de présence	Oui	Oui	Oui
Supervision du volet décisionnel	Oui	Non	Non
Consultation du journal d'audit	Oui	Non	Non

Ce cloisonnement garantit qu'un utilisateur ne peut accéder qu'aux fonctionnalités autorisées par son rôle : une tentative d'accès direct à une ressource réservée est bloquée par le contrôle de session et de rôle.

3.5 Diagrammes de séquence

Les diagrammes de séquence [4] décrivent le déroulement dynamique des principales fonctionnalités, en montrant les échanges entre la couche de présentation, les servlets, les services métier et la base de données.

3.5.1 S'authentifier

La figure 3.6 décrit le déroulement dynamique de l'authentification.

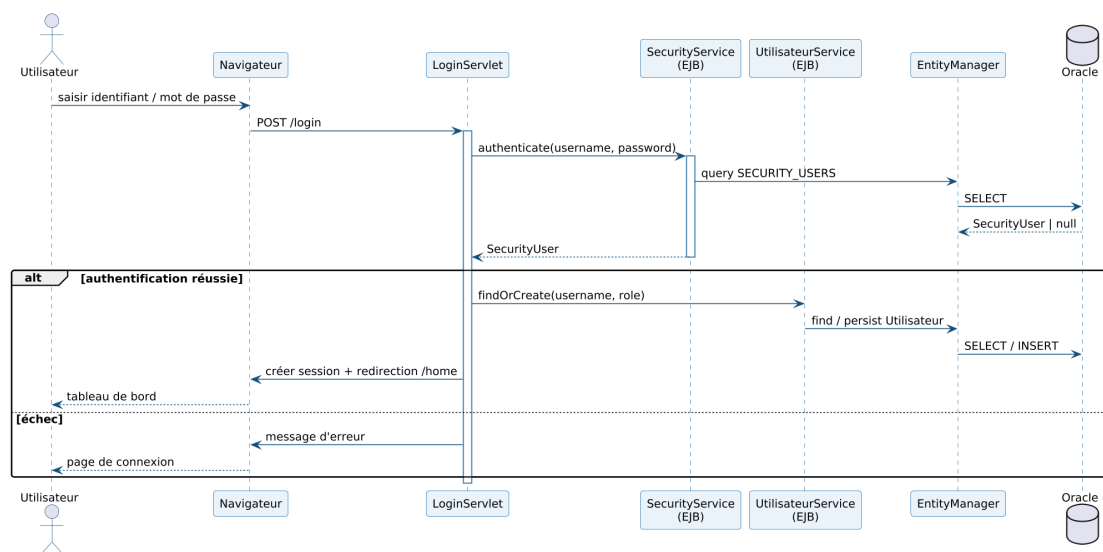


Figure 3.6 – Diagramme de séquence – Authentification

Le LoginServlet reçoit la requête POST /login et délègue la vérification au SecurityService, qui interroge la table SECURITY_USERS via l'EntityManager. Le fragment combiné alt distingue les deux issues : en cas de succès, l'Utilisateur métier est associé (ou créé), une session est ouverte et l'utilisateur est redirigé vers le tableau de bord ; en cas d'échec, un message d'erreur est renvoyé.

3.5.2 Créer une agence

La figure 3.7 décrit le scénario de création d'une agence.

L'AgenceServlet reçoit la requête de création et délègue à l'AgenceService, qui persiste la nouvelle Agence via l'EntityManager (INSERT), puis recharge la liste des agences renvoyée

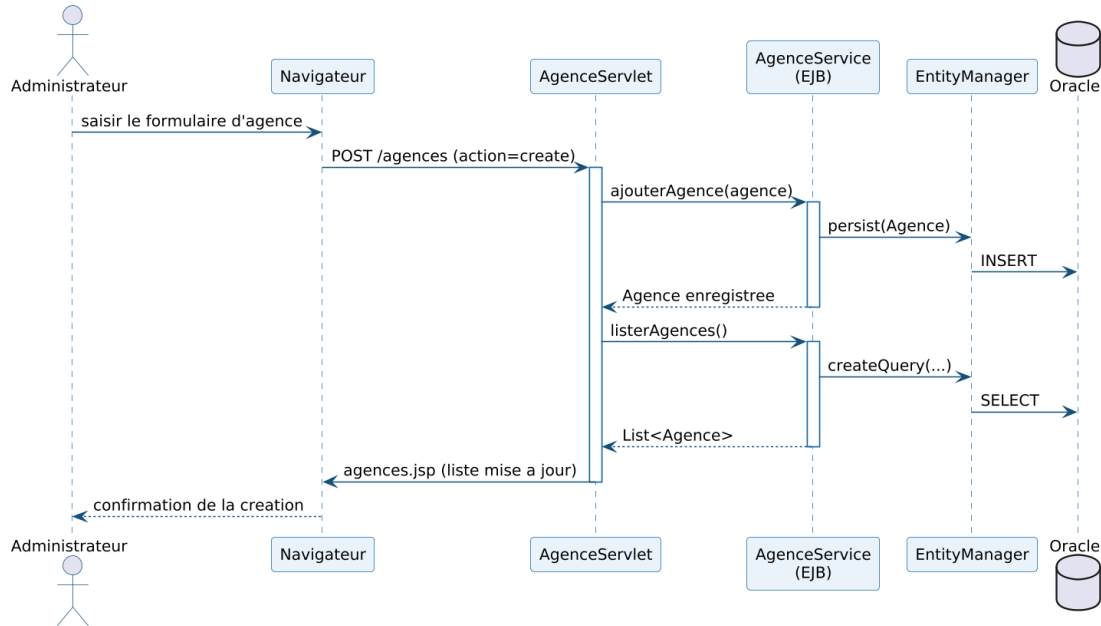


Figure 3.7 – Diagramme de séquence – Création d'une agence

à la page `agences.jsp`. Le même schéma s'applique aux opérations de modification et de suppression.

3.5.3 Gérer un client

La figure 3.8 décrit l'enregistrement et l'affectation d'un client.

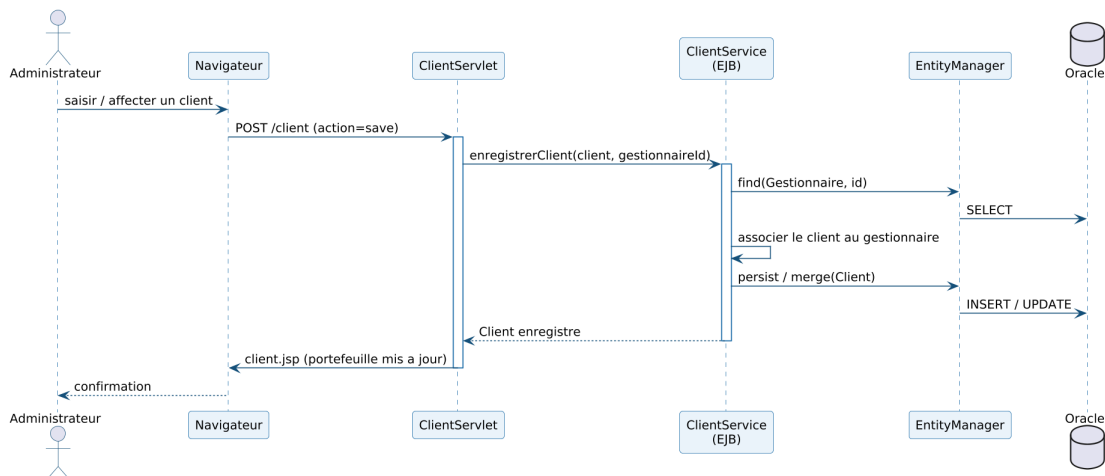


Figure 3.8 – Diagramme de séquence – Gestion d'un client

Le `ClientServlet` transmet la demande au `ClientService`, qui recherche le `Gestionnaire` concerné, associe le `Client` à ce dernier, puis le persiste via l'`EntityManager`. Le portefeuille mis à jour est renvoyé à la page `client.jsp`.

3.5.4 Pointer un employé

La figure 3.9 décrit le scénario de pointage.

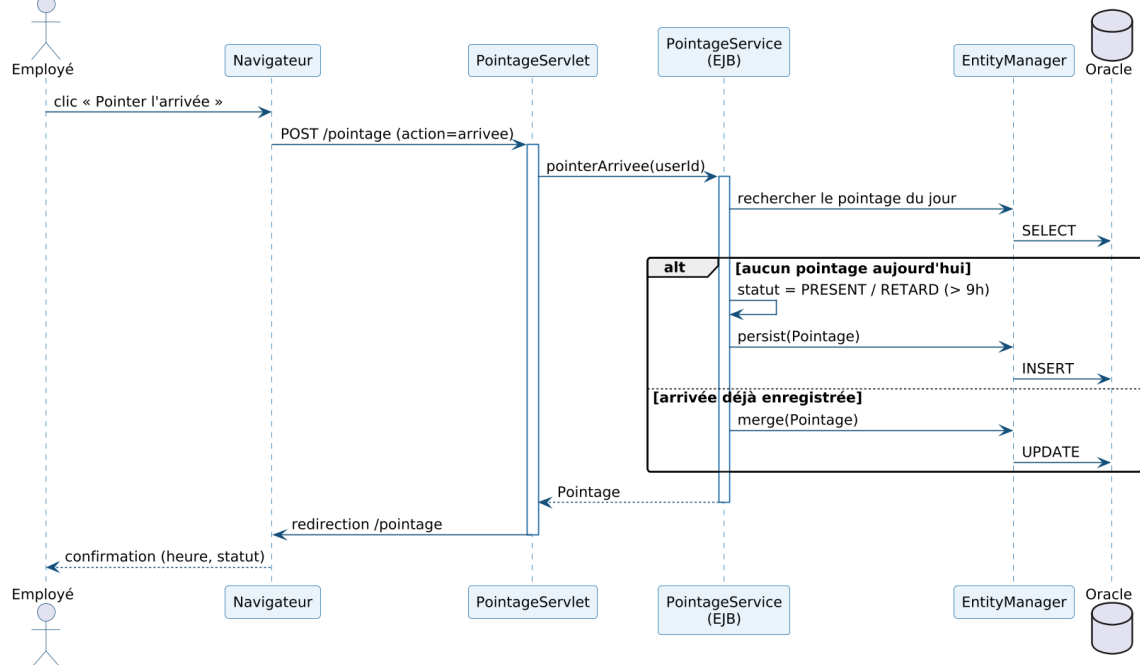


Figure 3.9 – Diagramme de séquence – Pointage d'un employé

Le PointageServlet transmet l'action au PointageService, qui recherche le pointage du jour. Le fragment alt distingue l'INSERT (premier pointage, statut PRESENT/RETARD au-delà de 9 h) de l'UPDATE (pointage déjà existant).

3.5.5 Valider une demande

La figure 3.10 détaille le traitement d'une demande par l'administrateur.

Le GestionModificationsServlet transmet la décision au service métier, qui charge la demande, met à jour son statut, persiste la modification puis journalise l'action via le JournalService.

3.6 Cycle de vie d'une demande

Les circuits de validation constituent un mécanisme transverse important. Leur comportement est modélisé par un diagramme d'états-transitions et un diagramme d'activité.

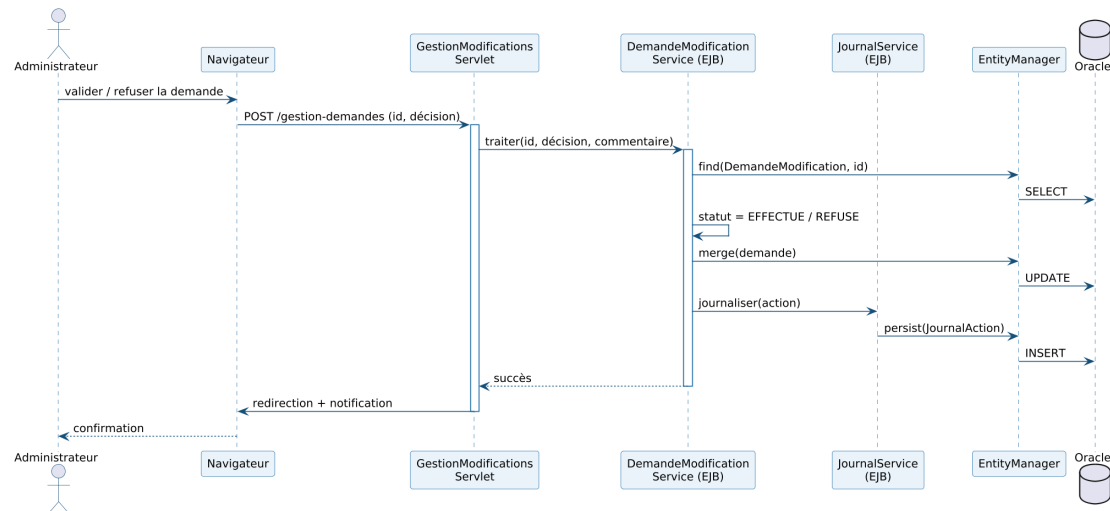


Figure 3.10 – Diagramme de séquence – Validation d’une demande

3.6.1 Diagramme d’états-transitions

La figure 3.11 modélise le cycle de vie d’une demande.

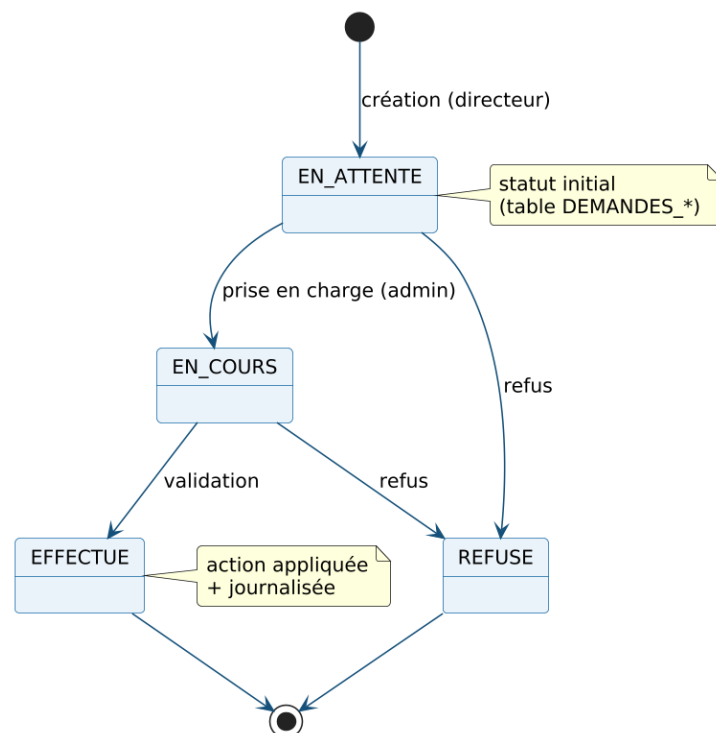


Figure 3.11 – Diagramme d’états-transitions – Cycle de vie d’une demande

Une demande naît à l’état EN_ATTENTE, passe en EN_COURS lors de sa prise en charge, puis atteint un état final EFFECTUE ou REFUSE, en cohérence avec la contrainte de la table des demandes.

3.6.2 Diagramme d'activité

La figure 3.12 représente le processus métier de traitement d'une demande.

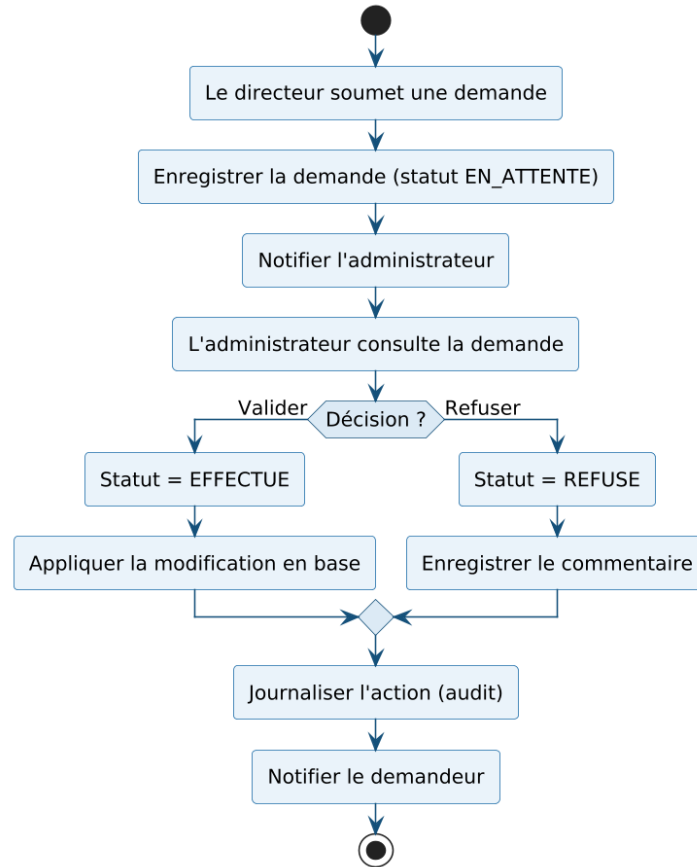


Figure 3.12 – Diagramme d'activité – Traitement d'une demande

Le directeur soumet une demande, notifiée à l'administrateur. Le nœud de décision oriente le flux : en cas de validation, le statut passe à EFFECTUE et la modification est appliquée; en cas de refus, le statut passe à REFUSE avec un commentaire. Les deux branches convergent vers la journalisation et la notification du demandeur.

Conclusion

Ce chapitre a défini l'architecture du système, son modèle de classes, son schéma relationnel, ses mécanismes de sécurité et les scénarios dynamiques de ses fonctionnalités clés. Le chapitre suivant présente le **volet décisionnel** : la chaîne ETL, le modèle en étoile, les tableaux de bord et la segmentation des agences.

Chapitre 4

Volet décisionnel

Introduction

Ce chapitre présente le **volet décisionnel** de la solution. Il met en place une chaîne **ETL** qui consolide les données opérationnelles dans un **datamart**, restitue les indicateurs au moyen d'un tableau de bord embarqué et de rapports Power BI, puis introduit une brique d'**intelligence artificielle** : la segmentation des agences par apprentissage non supervisé.

4.1 Chaîne décisionnelle et processus ETL

Les données nécessaires au pilotage sont dispersées dans les tables opérationnelles et exprimées à des grains hétérogènes. Le processus **ETL** (*Extract – Transform – Load*) les consolide en un datamart agrégé par agence. La figure 4.1 présente cette chaîne décisionnelle.

Trois étapes s'enchaînent : **Extract** (lecture des sources AGENCE, B_UTILISATEURS, CLIENT_BTK, B_OBJECTIF, POINTAGE), **Transform** (nettoyage, typage et agrégation par agence) et **Load** (chargement du datamart en étoile). Ce datamart constitue une **source unique consolidée**, partagée par la restitution (tableau de bord et Power BI) et par la segmentation des agences présentée à la section 4.7.

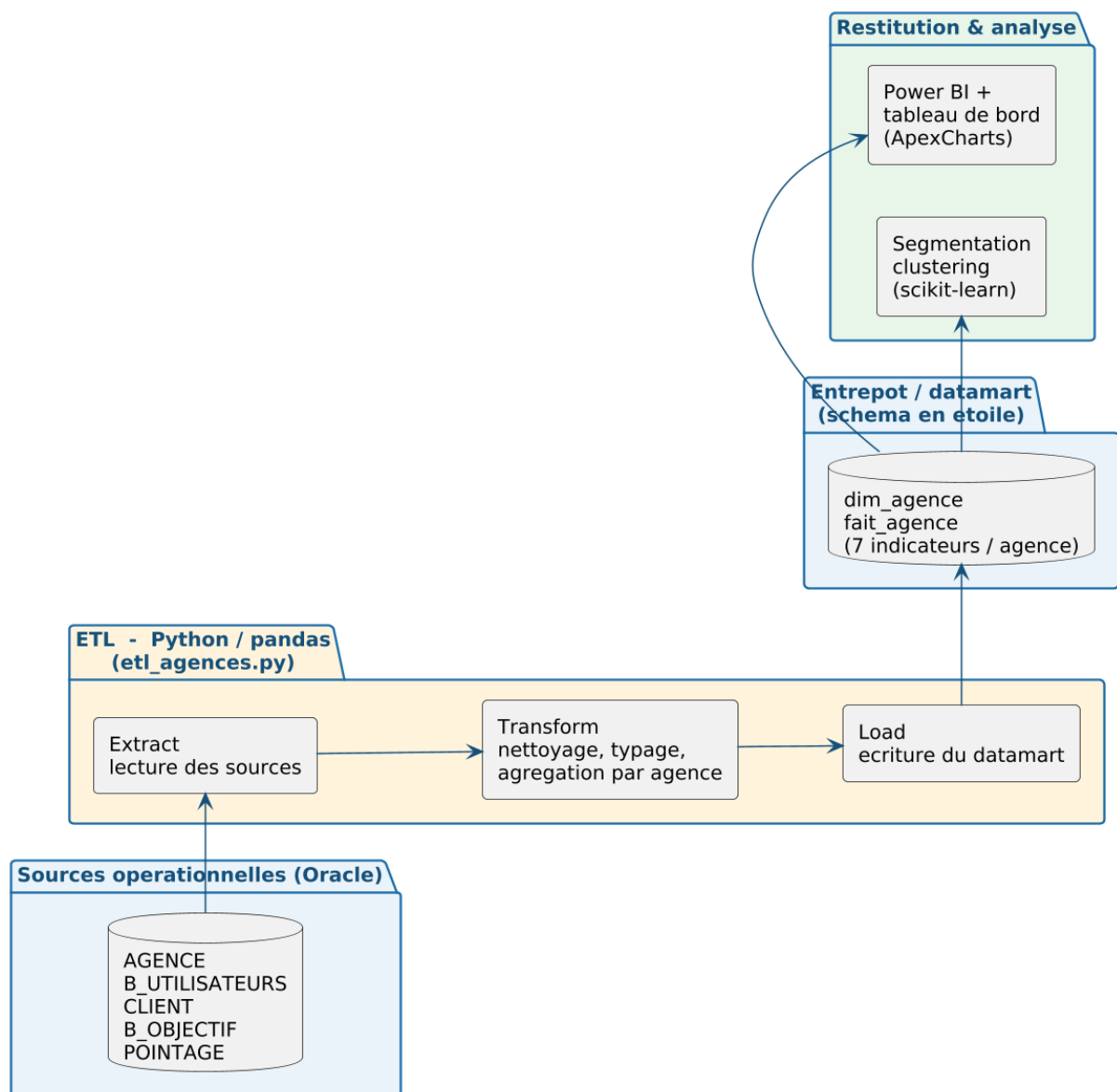


Figure 4.1 – Chaîne décisionnelle : du système opérationnel au datamart et à sa restitution

4.2 Modèle en étoile de l'entrepôt

4.2.1 Comparaison des schémas de modélisation

La modélisation dimensionnelle d'un entrepôt de données [11] peut suivre trois grands schémas. Le **modèle en étoile** place une table de faits centrale, entourée de tables de dimensions *dénormalisées* (une table par axe d'analyse). Le **modèle en flocon** (*snowflake*) en dérive : les dimensions y sont *normalisées* en sous-tables hiérarchiques, ce qui économise l'espace mais multiplie les jointures. Le **modèle en constellation** (*galaxy*) comporte enfin *plusieurs* tables de faits partageant des dimensions communes, afin de couvrir plusieurs processus métier au sein

d'un même entrepôt. Le tableau 4.1 confronte ces trois approches.

Table 4.1 – Comparaison des schémas de modélisation des entrepôts de données

Schéma	Principe	Avantages / limites
Étoile	Une table de faits centrale, dimensions dénormalisées (une table par axe).	+ Simple, lisible, requêtes rapides (peu de jointures). – Légère redondance dans les dimensions.
Flocon	Dimensions normalisées en sous-tables hiérarchiques.	+ Moins de redondance, espace optimisé. – Requêtes plus complexes, plus de jointures, lisibilité réduite.
Constellation	Plusieurs tables de faits partageant des dimensions communes.	+ Couvre plusieurs processus métier. – Modèle plus lourd, réservé aux entrepôts de grande ampleur.

Pour ce projet, le **modèle en étoile** a été retenu. Le besoin porte sur un **grain unique et homogène** — l'agence — et sur un **nombre réduit de dimensions** (agence, période, gestionnaire, type de client) : la normalisation en flocon n'apporterait ici aucun gain significatif d'espace tout en alourdissant les requêtes, et la constellation, pensée pour plusieurs processus de faits, serait surdimensionnée. L'étoile offre en revanche la **simplicité**, la **rapidité des requêtes** et surtout la **lisibilité** attendues d'un outil de restitution en libre-service comme **Power BI**, où des relations directes entre la table de faits et ses dimensions garantissent un filtrage croisé immédiat.

4.2.2 Structure du modèle retenu

L'entrepôt est organisé selon une logique en **étoile** (figure 4.2) : autour de tables de faits (production, effectifs, clients, présence), des dimensions descriptives (agence, gestionnaire, période, type de client) permettent l'analyse multidimensionnelle. Les vues V_BI_* matérialisent cette couche sémantique pour l'outil de restitution.

Au centre du schéma, la **table de faits** porte les mesures numériques agrégées ; autour d'elle, les **tables de dimensions** fournissent les axes d'analyse. Le tableau 4.2 en précise le rôle.

Le tableau 4.3 récapitule les principaux indicateurs (KPI) retenus.

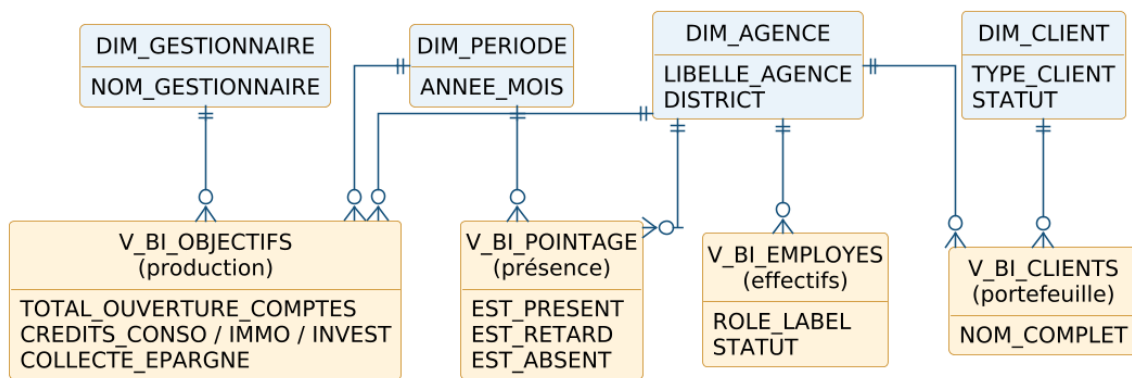


Figure 4.2 – Modèle décisionnel en étoile (vues V_BI_*)

Table 4.2 – Table de faits et tables de dimensions du modèle en étoile

Rôle	Table	Description
Table de faits	fait_agence	Mesures agrégées par agence : effectif, nombre de gestionnaires, portefeuille clients, comptes ouverts, production de crédits, collecte d'épargne et taux de présence.
Dimension	dim_agence	Axe « agence » : libellé et district — analyse géographique par agence et par région.
Dimension	période	Axe temporel (ANNEE_MOIS) : analyse par mois et par exercice.
Dimension	gestionnaire	Axe « responsable » : analyse par gestionnaire de portefeuille.
Dimension	type_client	Axe « portefeuille » : analyse par type de client (particulier, professionnel, entreprise).

Table 4.3 – Indicateurs de performance (KPI) du volet décisionnel

Indicateur	Description	Source
Effectifs par agence	Nombre d'employés par agence	V_BI_EMPLOYES
Répartition des rôles	Gestionnaires / conseillers / hors commercial	V_BI_EMPLOYES
Portefeuille clients	Clients par type et par agence	V_BI_CLIENTS
Ouvertures de comptes	Comptes chèques + épargne + courants	V_BI_OBJECTIFS
Production de crédits	Conso, immobilier, investissement	V_BI_OBJECTIFS
Collecte d'épargne	Épargne additionnelle collectée	V_BI_OBJECTIFS
Taux de présence	Présents, retards et absences par agence et période	V_BI_POINTAGE

4.3 Mise en œuvre de la chaîne ETL en Python

4.3.1 Export des données sources

La chaîne ETL peut se connecter directement à Oracle ou, à défaut, consommer des fichiers CSV. Pour ce second mode, un script **SQLcl** (`etl/export_sources.sql`) exporte les cinq tables sources — agences, employés, clients, objectifs et pointages — vers des fichiers CSV, placés ensuite dans le répertoire `etl/source/`. Les indicateurs d'objectifs (comptes, crédits, épargne) y sont déjà consolidés à partir des colonnes correspondantes de la table `B_OBJECTIF`.

4.3.2 Détail des étapes de traitement

La chaîne ETL est implémentée en **Python** (bibliothèque **pandas** [12]) et exécutée dans un **notebook** connecté à la base Oracle. Elle enchaîne cinq étapes — **collecte**, **nettoyage**, **transformation**, **intégration** et **contrôle de qualité** —, illustrées ci-après par des captures du notebook.

1. Collecte (Extract). Les cinq jeux de données sources — agences, employés, clients, objectifs et pointages — sont lus depuis les tables Oracle `AGENCE`, `B_UTILISATEURS`, `CLIENT_BTK`, `B_OBJECTIF` et `POINTAGE` (via le connecteur `oracledb`). La figure 4.3 en présente l'exécution.

```
[1] # 1) COLLECTE (Extract) : lecture des tables sources Oracle
agences = q("SELECT SK_AGENCE, LIBELLE_AGENCE, DISTRICT FROM AGENCE")
employes = q("SELECT SK_UTILISATEUR, SK_AGENCE, EST_GESTIONNAIRE FROM B_UTILISATEURS")
clients = q("SELECT SK_CLIENT, SK_AGENCE FROM CLIENT_BTK")
objectifs = q("SELECT SK_AGENCE, COMPTES, CREDITS, EPARGNE FROM B_OBJECTIF")
print("Agences:", len(agences), "| Employes:", len(employes), "| Clients:", len(clients))
agences.head(10)
```

Agences: 49 | Employes: 997 | Clients: 29942

SK_AGENCE	LIBELLE_AGENCE
1	BIZERTE
2	MGHIRA
3	MONASTIR
4	GROMBALIA
5	BEN AROUS
6	CENTRALE
7	NABEUL
8	SFAX 2
9	LA MARSA
10	PALMARIUM

Figure 4.3 – Collecte : lecture des tables sources du réseau BTK (données réelles)

2. Nettoyage. Les enregistrements incomplets ou incohérents sont écartés : les lignes sans agence de rattachement (`SK_AGENCE` manquant) sont supprimées, puis les colonnes sont **typées** (entiers, décimaux, libellés) afin de fiabiliser les calculs ultérieurs.

3. Transformation. C'est le cœur du traitement : un regroupement (*group by*) par agence calcule, pour chacune, les **sept indicateurs** de pilotage. Le tableau 4.4 précise le mode de calcul de chaque indicateur.

Table 4.4 – Calcul des indicateurs lors de l'étape de transformation

Indicateur	Calcul (agrégation par agence)
effectif	Nombre d'employés rattachés à l'agence.
nb_gestionnaires	Somme des employés marqués gestionnaires.
nb_clients	Nombre de clients du portefeuille de l'agence.
total_comptes	Somme des ouvertures de comptes (chèques, épargne, courants).
production_credits	Somme de la production de crédits (conso, immobilier, investissement).
collecte_epargne	Somme de l'épargne additionnelle collectée.
taux_presence	Proportion de pointages « présent » ou « retard » sur le total.

4. Intégration. Les indicateurs agrégés issus des différentes sources (effectifs, portefeuille, production, présence) sont **fusionnés** sur la clé `SK_AGENCE` en une table unique décrivant chaque agence. La figure 4.4 présente cette agrégation sur les **données réelles du réseau BTK** (49 agences) : elle détaille, pour chaque agence, l'effectif, le nombre de gestionnaires, le portefeuille clients et la production commerciale (comptes, crédits, épargne). Le taux de présence est calculé de la même manière à partir de la table `POINTAGE`.

5. Contrôle de qualité. Avant chargement, la qualité du datamart est vérifiée : les **valeurs manquantes** sont remplacées par zéro et les mesures sont **arrondies**. Un contrôle final confirme l'**absence de valeur manquante** et récapitule les statistiques du datamart (figure 4.5).

Chargement (Load). Le datamart en étoile est enfin écrit sous `etl/entrepot/` : la dimension `dim_agence` et la table de faits `fait_agence`. Un fichier consolidé (`clustering/data/agences.csv`) est également produit pour alimenter la segmenta-


```
[2] # 3) TRANSFORMATION + 4) INTEGRATION : agregation par agence
eff = employees.groupby('SK_AGENCE').agg(
    effectif=('SK_UTILISATEUR', 'count'),
    nb_gestionnaires=('EST_GESTIONNAIRE', 'sum')).reset_index()
cli = clients.groupby('SK_AGENCE').size().reset_index(name='nb_clients')
obj = objectifs.groupby('SK_AGENCE').agg(
    total_comptes=('COMPTES', 'sum'),
    production_credits=('CREDITS', 'sum'),
    collecte_epargne=('EPARGNE', 'sum')).reset_index()
datamart = agences.merge(eff).merge(cli).merge(obj).rename(
    columns={'LIBELLE_AGENCE': 'agence'})
datamart.sort_values('nb_clients', ascending=False).head(10)
```

	agence	effectif	nb_gestionnaires	nb_clients	total_comptes	production_credits	collecte_epargne
0	BIZERTE	10	8	1795	799	6300000	812500
1	MGHIRA	9	6	1791	607	5880000	690000
2	MONASTIR	12	10	1497	557	7000000	450000
3	GROMBALIA	12	6	1451	618	5400000	662500
4	BEN AROUS	18	7	1313	708	8300000	720000
5	CENTRALE	26	9	1226	528	59400000	420000
6	NABEUL	8	7	1225	669	6350000	840000
7	SFAX 2	15	6	1154	436	4750000	385000
8	LA MARSIA	8	5	1126	618	6250000	662500
9	PALMARJUM	13	5	1118	712	8100000	732500

Figure 4.4 – Datamart par agence sur les données réelles du réseau BTK (49 agences)

```
[3] # 5) CONTROLE DE QUALITE : valeurs manquantes + statistiques
print('Valeurs manquantes : ', int(datamart[COLS[1:]].isna().sum().sum()))
print('Datamart : ', datamart.shape[0], 'agences x', len(COLS), 'colonnes')
datamart.describe().round(0)
```

Valeurs manquantes : 0
Datamart : 49 agences x 7 colonnes

	stat	effectif	nb_gestionnaires	nb_clients	total_comptes	production_credits	collecte_epargne
0	count	49	49	49	49	49	49
1	mean	20	7	611	428	7718138	478939
2	std	76	10	566	252	10332885	517199
3	min	0	0	0	0	0	0
4	25%	5	4	2	333	3800000	210000
5	50%	10	6	724	502	5324000	457500
6	75%	14	8	1058	607	6500000	650000
7	max	540	76	1795	801	59400000	3508025

Figure 4.5 – Contrôle de qualité : statistiques du datamart (données réelles du réseau BTK)

tion. Le pipeline étant **rejouable**, il bascule automatiquement sur les données réelles dès qu'un export est fourni.

4.4 Tableau de bord et restitution Power BI

Le tableau de bord embarqué (`home.jsp`, alimenté par le `HomeServlet`) comporte une rangée de **cartes d'indicateurs** (employés, clients, présents du jour, taux de présence, demandes en attente) puis des **graphiques** ApexCharts (effectif par agence, répartition des rôles, présence du jour). En complément, les vues `V_BI_*` sont connectées à **Power BI** pour des rapports interactifs. La figure 4.6 présente le tableau de bord embarqué.

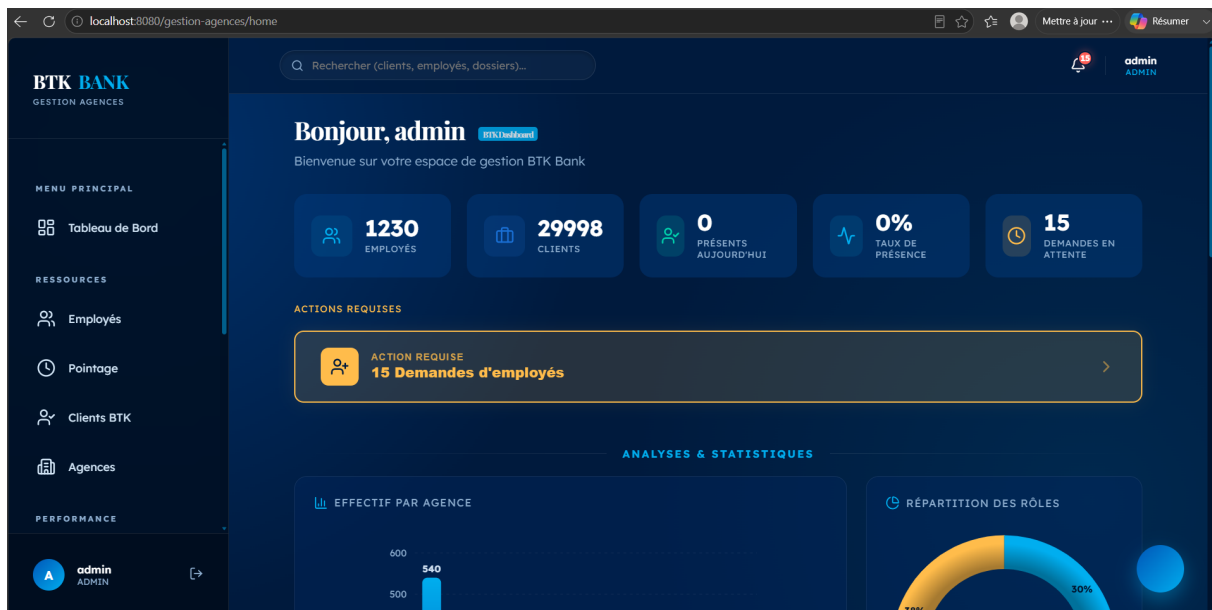


Figure 4.6 – Tableau de bord décisionnel embarqué de l'application

4.5 Intégration du modèle en étoile dans Power BI

Le modèle en étoile décrit précédemment est **matérialisé dans Power BI** [7] pour servir de socle aux rapports. Les tables et vues décisionnelles sont d'abord **importées** depuis le schéma Oracle (AGENCE, B_UTILISATEURS, B_CLIENTS, B_OBJECTIF, POINTAGE et les vues V_BI_*) au moyen du connecteur Oracle natif.

4.5.1 Création des relations entre les tables

Dans la *vue Modèle* de Power BI, des **relations** sont établies entre la dimension centrale AGENCE et les tables de faits, sur les clés de substitution (SK_AGENCE et SK_UTILISATEUR). La figure 4.7 présente le modèle obtenu.

Chaque relation est de type **un-à-plusieurs** (*one-to-many*), orientée de la dimension vers les faits, avec un filtre à **sens unique**. Le tableau 4.5 en récapitule le détail.

4.5.2 Exploitation dans les visualisations

Ces relations sont le **moteur de l'interactivité** du rapport. Le filtre se propageant du côté « 1 » vers le côté « plusieurs » (de la dimension vers les faits), la sélection d'une agence, d'un district ou d'une période dans un segment — ou dans n'importe quel visuel — **recalcule instantanément**

Modèle Power BI — relations en étoile autour de la dimension AGENCE

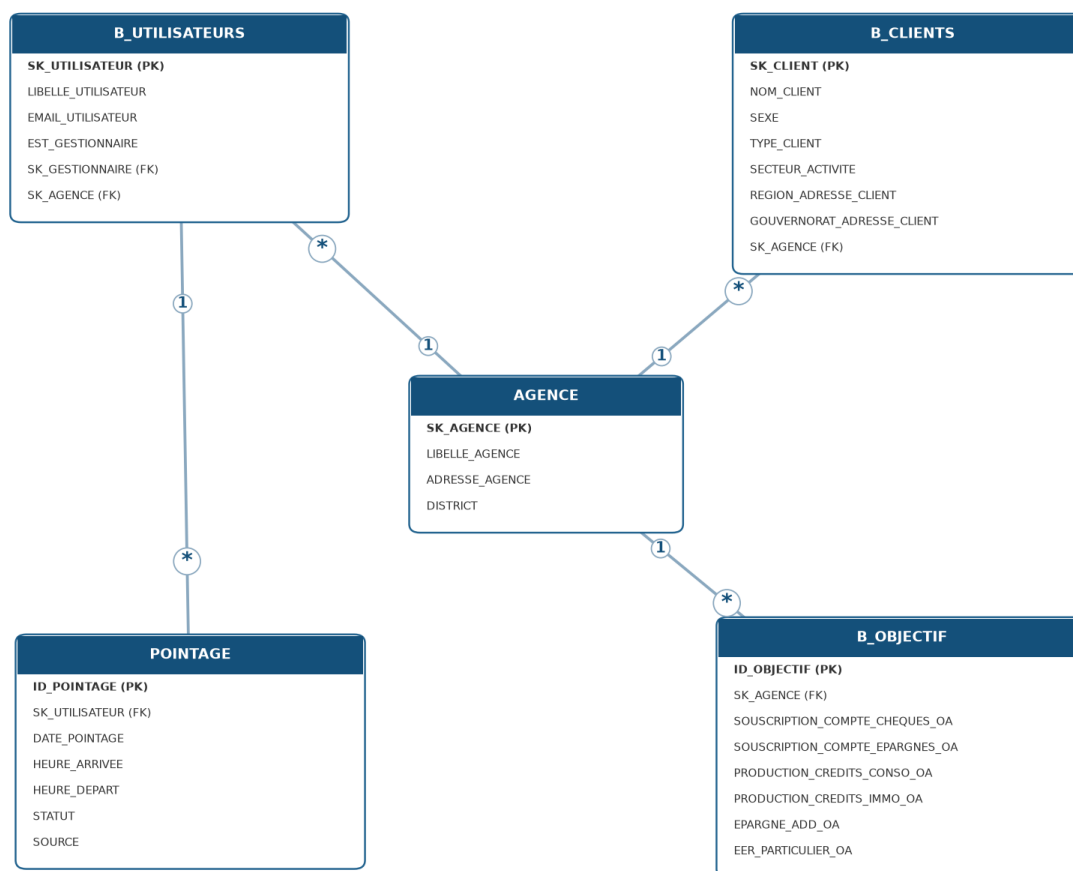


Figure 4.7 – Modèle de données Power BI : relations en étoile autour de la dimension AGENCE

Table 4.5 – Relations du modèle Power BI et leur cardinalité

Table (côté 1)	Table (côté *)	Clé de jointure	Cardinalité	
AGENCE	B_UTILISATEURS	SK_AGENCE	1 – *	
AGENCE	B_CLIENTS	SK_AGENCE	1 – *	
AGENCE	B_OBJECTIF	SK_AGENCE	1 – *	
B_UTILISATEURS	POINTAGE	SK_UTILISATEUR	1 – *	

tous les visuels reliés (*filtrage croisé*). Les mesures DAX présentées à la section suivante s'appuient sur ce modèle : elles agrègent les faits en respectant les relations, ce qui garantit la cohérence des indicateurs quel que soit l'axe d'analyse retenu (agence, période, type de client). C'est cette intégration du modèle en étoile qui fait passer Power BI d'un simple afficheur de tableaux à un véritable outil d'**analyse multidimensionnelle**.

4.6 Rapport décisionnel sous Power BI

Au-delà du tableau de bord embarqué, un **rapport décisionnel dédié** a été développé sous **Microsoft Power BI**, connecté aux données consolidées. Il est organisé en **quatre pages** analytiques thématiques — *Résumé*, *Clients*, *Commercial* et *Présence* — reliées par une barre de navigation, chacune munie de **segments** de filtrage (district, période, type de client) qui recalculent l'ensemble des visuels d'un simple clic. La **page de synthèse** (*Résumé*) regroupe les indicateurs clés du réseau (clients, production de crédits, taux de présence, ouvertures de comptes), la répartition des rôles et des présences, l'évolution mensuelle de la production et un classement des agences. Les trois pages suivantes approfondissent respectivement l'analyse des clients, la performance commerciale et l'assiduité.

4.6.1 Indicateurs et mesures (DAX)

Les indicateurs restitués sont calculés au moyen de **mesures DAX** définies sur le modèle de données. Le tableau 4.6 en récapitule les principales.

Au-delà de leur définition, ces indicateurs sont retenus pour l'**aide à la décision** qu'ils apportent. Le tableau 4.7 précise l'**intérêt décisionnel** de chacun.

4.6.2 Page de synthèse

La page *Résumé* (figure 4.8) offre une vue d'ensemble du réseau. Une rangée de **cartes d'indicateurs** présente les chiffres clés (nombre de clients, production de crédits, taux de présence, ouvertures de comptes) ; deux **anneaux** donnent la répartition des présences et des rôles ; un **histogramme** retrace l'évolution mensuelle de la production de crédits et un **tableau**

Table 4.6 – Principales mesures DAX du rapport Power BI

Mesure	Définition
Nb Employés	Nombre distinct d'employés (SK_UTILISATEUR).
Nb Clients	Nombre distinct de clients (SK_CLIENT).
Total Comptes	Somme des ouvertures de comptes (chèques + épargne + courants).
Total Crédits	Somme de la production de crédits (conso + immobilier + investissement).
Total Épargne	Épargne additionnelle collectée.
Total Packs & Cartes	Souscriptions de packs (particulier, pro) et de cartes.
Taux Présence	Part des pointages au statut « présent » ou « retard ».
Nb Présences / Retards / Absences	Nombre de pointages par statut.
Rôle (colonne calculée)	Gestionnaire / Conseiller / Hors commercial, selon EST_GESTIONNAIRE et l'agence.

Table 4.7 – Principaux KPI des tableaux de bord et leur intérêt décisionnel

KPI	Intérêt décisionnel
Nombre de clients	Taille du portefeuille : dimensionne l'activité et l'allocation des ressources par agence.
Production de crédits	Performance commerciale majeure et source de revenus : suivi de l'objectif prioritaire.
Total des comptes ouverts	Effort de conquête et d'équipement : mesure la croissance du portefeuille.
Collecte d'épargne	Capacité à mobiliser des ressources : équilibre entre emplois et ressources.
Taux de présence	Assiduité des équipes : disponibilité commerciale et suivi RH.
Répartition des rôles	Structure des effectifs : adéquation entre gestionnaires et conseillers.
Segmentation des agences	Profils d'agences : ciblage des actions et fixation d'objectifs différenciés.

classe les agences selon leur performance. Les trois **segments** de l'en-tête (district, période, type de client) et le bouton **RÉINITIALISER** pilotent l'ensemble.

Interprétation. Le réseau compte environ **30 000 clients** et **105 000 comptes**, pour une production de crédits de **1,40 milliard** et un **taux de présence de 87,9 %**. La répartition des présences révèle toutefois que près d'un pointage sur trois est un **retard** (25 %) ou une **absence** (12 %) : l'assiduité apparaît ainsi comme un premier levier d'amélioration. Les effectifs se partagent de façon relativement équilibrée entre personnel hors commercial (38 %), conseillers (32 %) et gestionnaires (30 %). L'évolution mensuelle met en évidence un **pic de production en février** suivi d'un tassement, que le décideur peut interpréter comme un effet de campagne à reconduire. Enfin, le classement des agences traduit une **forte hétérogénéité** du réseau — que

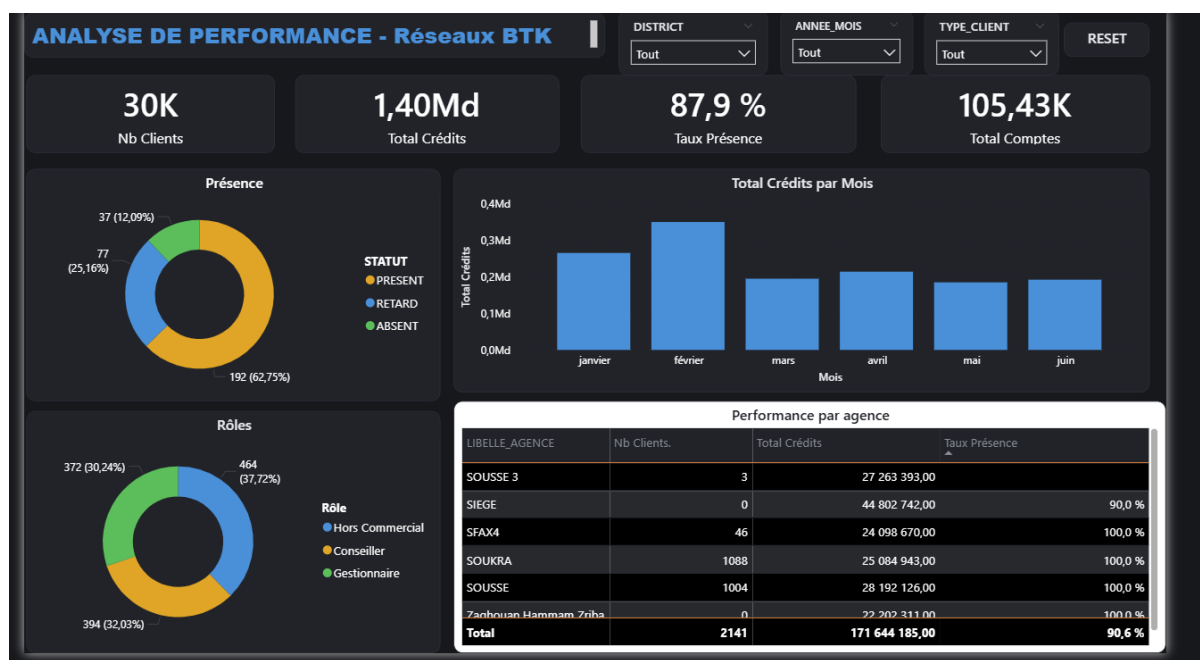


Figure 4.8 – Rapport Power BI — page « Résumé » (synthèse du réseau)

les trois pages suivantes approfondissent.

4.6.3 Analyse des clients

La page *Clients* (figure 4.9) segmente le portefeuille selon plusieurs axes : le **type de client**, le **sexe**, la **région** et le **gouvernorat** (histogramme des principaux gouvernorats), complétés par un classement des agences selon leur nombre de clients. Elle offre une lecture **géographique et démographique** du portefeuille, absente du tableau de bord opérationnel.

Interprétation. Le portefeuille est très majoritairement composé de **particuliers** (88 %), les segments *société* et *entreprise* ne pesant qu'environ 12 % : il existe donc une réelle **marge de développement** sur la clientèle professionnelle, à plus forte valeur ajoutée. La clientèle est par ailleurs **masculine à 63 %** et fortement **concentrée sur le Grand Tunis** (Tunis, Ben Arous, Ariana), les régions Tunis Nord et Tunis Sud totalisant à elles seules plus de la moitié des clients. Ces constats invitent à **cibler les régions sous-représentées** (Centre, Sud) et à rééquilibrer la conquête ; le classement des agences par nombre de clients aide, quant à lui, à prioriser les efforts commerciaux.

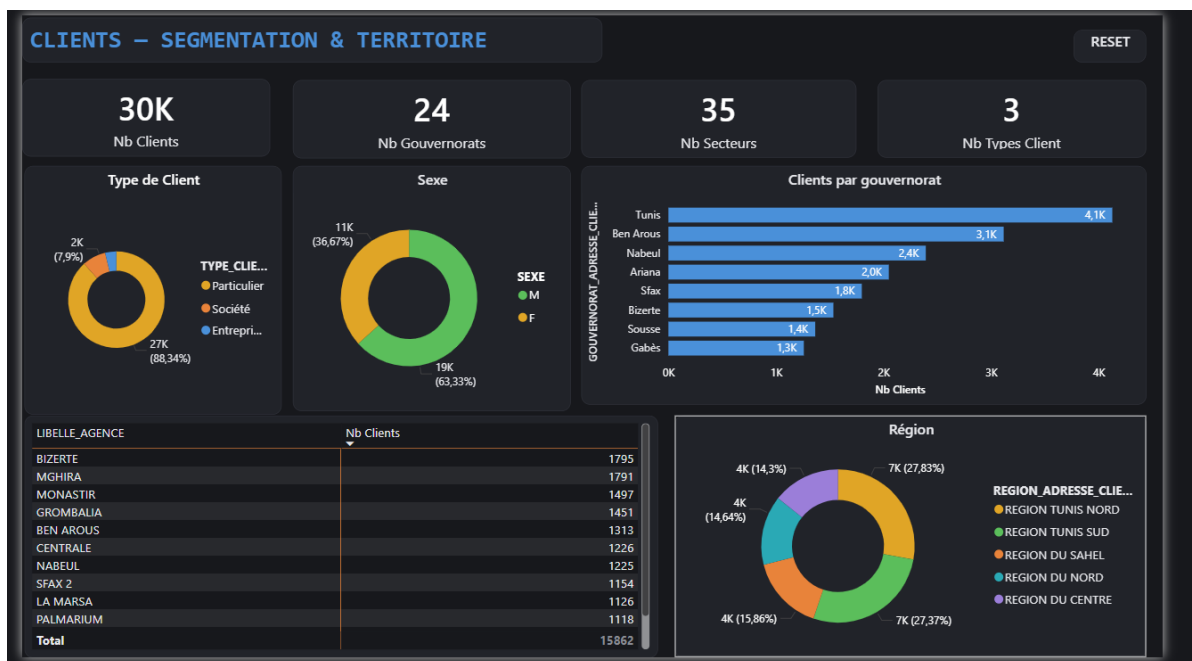


Figure 4.9 – Rapport Power BI — page « Clients » (segmentation et territoire)

4.6.4 Performance commerciale

La page *Commercial* (figure 4.10) détaille les **objectifs** : les ouvertures de comptes par type, la production de crédits par nature, l'effort d'équipement (EER), l'**évolution mensuelle** de la production et un classement des agences par production. Elle constitue l'outil de **pilotage commercial** du réseau.

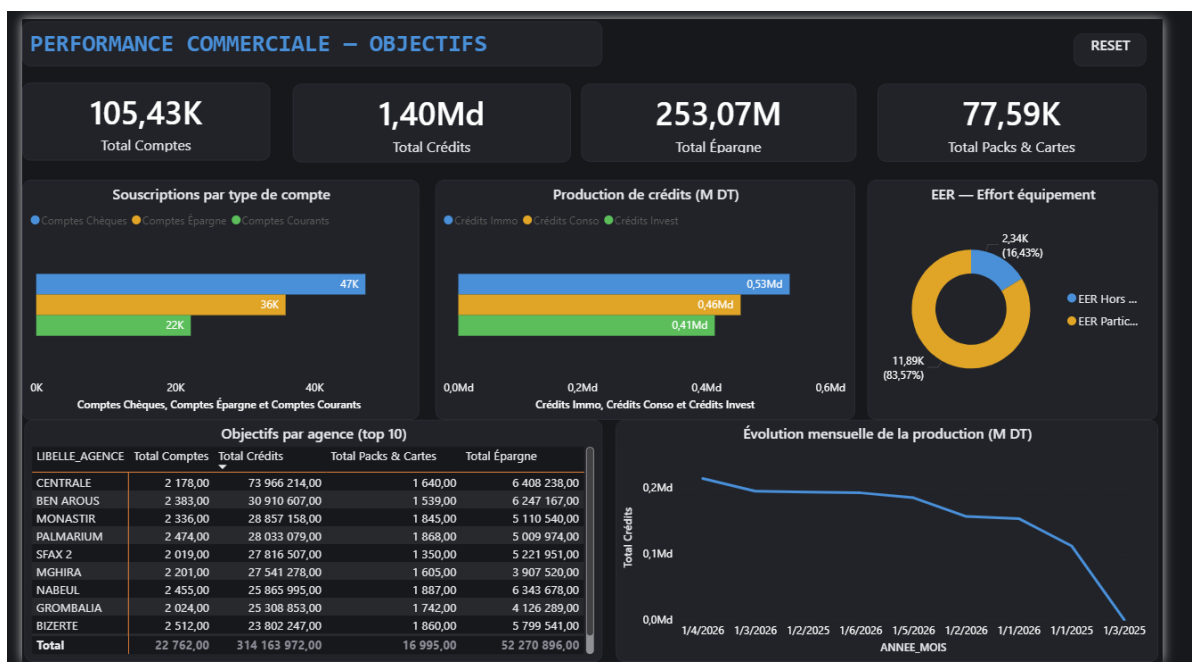


Figure 4.10 – Rapport Power BI — page « Commercial » (objectifs et production)

Interprétation. La production de crédits (1,40 milliard) se répartit de manière assez **équilibrée** entre l'immobilier (0,53 Md), la consommation (0,46 Md) et l'investissement (0,41 Md), l'immobilier restant en tête. Côté équipement, les **comptes chèques** dominent les ouvertures (47 K, devant l'épargne et les comptes courants) et l'effort d'équipement (EER) porte à **83,6 % sur les particuliers** : la clientèle professionnelle constitue là encore un gisement de croissance. Le classement des agences révèle enfin une agence phare, **CENTRALE**, dont la production dépasse largement celle des suivantes ; ses bonnes pratiques peuvent être diffusées au réseau. Le décideur dispose ainsi d'une base factuelle pour **diversifier la production** vers l'investissement et le segment entreprise.

4.6.5 Suivi de la présence

La page *Présence* (figure 4.11) suit l'**assiduité** des employés : le taux de présence, la répartition des statuts (présent, retard, absent) et de la source de pointage (automatique, manuel), l'**évolution mensuelle** du taux de présence et la liste des **agences à surveiller** (présence la plus faible). Elle prolonge le module de pointage par une vue **ressources humaines** consolidée.



Figure 4.11 – Rapport Power BI — page « Présence » (assiduité et suivi RH)

Interprétation. Avec un taux global de **87,9 %**, l'assiduité reste correcte mais perfectible : les **retards** représentent un quart des pointages (25 %) et les absences 12 %. Le pointage est majoritairement **automatique** (72 % en mode « auto » contre 28 % de saisies manuelles), signe

d'une bonne adoption de l'outil. L'évolution mensuelle décrit une **montée jusqu'à un pic en avril-mai** (proche de 92 %) suivie d'un **repli estival** vers juillet ($\approx 80\%$), tendance à surveiller. Surtout, la liste des **agences à surveiller** isole quelques établissements au taux critique (Montplaisir et Mourouj 6 à 50 %, Rades à 60 %) qui appellent une **action RH ciblée**. Cette page transforme ainsi le suivi du pointage en un véritable outil de **pilotage social** du réseau.

4.6.6 Navigation et interactivité

Les quatre pages sont reliées par un **navigateur** qui joue le rôle de menu. Chaque page est équipée de **segments** (district, période, type de client) et d'un bouton **RÉINITIALISER** : un clic sur une valeur recalcule instantanément l'ensemble des visuels de la page (*filtrage croisé*), et la sélection d'une part d'un graphique filtre à son tour les autres. Cette **interactivité** transforme le rapport en un véritable outil d'exploration : le décideur peut, par exemple, isoler un district puis observer simultanément le portefeuille, la production et l'assiduité correspondants. Un **thème sombre** homogène assure la lisibilité et la cohérence visuelle de l'ensemble.

4.7 Segmentation des agences par apprentissage non supervisé

Cette dernière section introduit une brique d'**intelligence artificielle** : la segmentation des agences par **apprentissage non supervisé** (*clustering*) [13]. À partir du datamart produit par la chaîne ETL, elle regroupe automatiquement les agences en profils homogènes afin d'appuyer le ciblage, l'allocation des ressources et la définition d'objectifs différenciés.

4.7.1 Données et variables

Les données ne sont pas extraites directement de la base opérationnelle : elles proviennent du **datamart** produit par la chaîne ETL (table de faits `fait_agence`). Chaque agence y est décrite par sept indicateurs agrégés (tableau 4.8), ce qui garantit la cohérence entre le tableau de bord, Power BI et l'analyse non supervisée.

Remarque. En l'absence d'un export des données de production, la segmentation présentée ici est **illustrée sur un jeu de données représentatif** du réseau, généré par la chaîne ETL pour reproduire la structure et les ordres de grandeur des agences ; le pipeline bascule automatiquement

sur les données réelles dès qu'un export est fourni (etl/source/). Les résultats ci-dessous illustrent donc la *démarche* et restent reproductibles sur les données réelles.

Table 4.8 – Variables de segmentation (par agence)

Variable	Description
effectif	Nombre d'employés de l'agence
nb_gestionnaires	Nombre de gestionnaires
nb_clients	Taille du portefeuille clients
total_comptes	Total des ouvertures de comptes
production_credits	Production de crédits
collecte_epargne	Collecte d'épargne
taux_presence	Taux de présence (issu du pointage)

4.7.2 Prétraitement

Les variables étant exprimées dans des ordres de grandeur très différents (un effectif de quelques dizaines face à des milliers de comptes), une **standardisation** (centrage-réduction, `StandardScaler` de la bibliothèque **scikit-learn** [14]) est appliquée. Sans elle, les variables de forte amplitude domineraient le calcul des distances et fausseraient le regroupement.

4.7.3 Détermination du nombre de clusters

Le nombre de clusters k étant inconnu a priori, il est déterminé en combinant la **méthode du coude** (inertie intra-cluster) et le **score de silhouette** (figure 4.12).

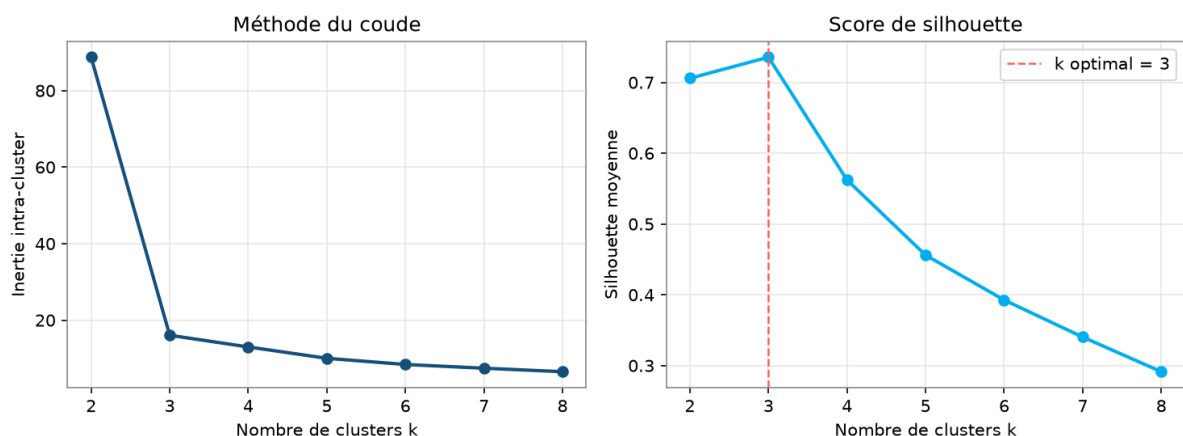


Figure 4.12 – Choix du nombre de clusters : méthode du coude et score de silhouette

Les deux indicateurs concordent : l'inertie présente un **coude marqué** à $k = 3$ et le score de silhouette y atteint son **maximum** ($\approx 0,74$). La segmentation en **trois clusters** est donc retenue.

4.7.4 Comparaison des modèles de clustering

Quatre algorithmes [13] ont été comparés : **K-Means**, **classification ascendante hiérarchique** (agglomératif), **modèle de mélange gaussien** (GMM) et **DBSCAN**, à l'aide de trois indices internes (tableau 4.9, figure 4.13).

Table 4.9 – Comparaison des modèles de clustering

Modèle	Nb clusters	Silhouette $\uparrow$	Davies-Bouldin $\downarrow$	Calinski-Harabasz $\uparrow$	
K-Means	3	0,736	0,350	391	
Agglomératif	3	0,736	0,350	391	
GMM	3	0,736	0,350	391	
DBSCAN	3	0,736	0,350	391	

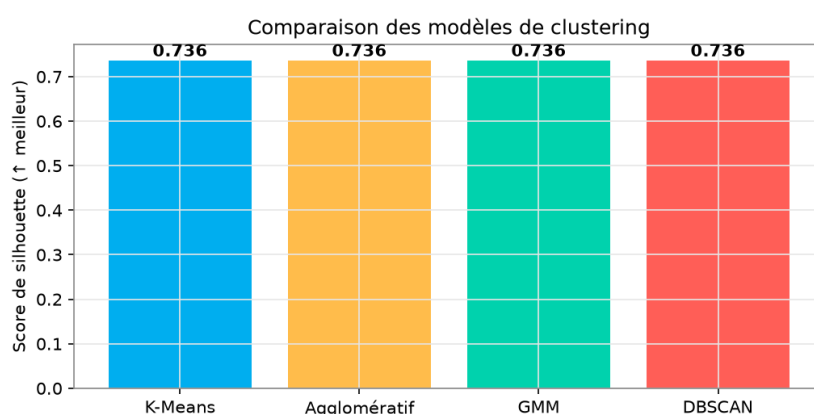


Figure 4.13 – Comparaison des modèles selon le score de silhouette

Les quatre algorithmes **convergent vers la même segmentation** : ils identifient les trois mêmes clusters et obtiennent des indices identiques (silhouette 0,736, Davies-Bouldin 0,350, Calinski-Harabasz 391). Cette concordance, y compris pour des approches de nature différente (partitionnement, hiérarchique, probabiliste et à base de densité), confirme la **robustesse** des trois segments. Le **K-Means** est retenu : il atteint le meilleur score tout en restant simple et interprétable.

4.7.5 Résultats et interprétation

La figure 4.14 projette les agences dans le plan des deux premières composantes principales (PCA), colorées selon leur cluster ; la première composante, qui résume la *taille* de l'agence, explique l'essentiel de la variance. Les trois segments sont nettement séparés.

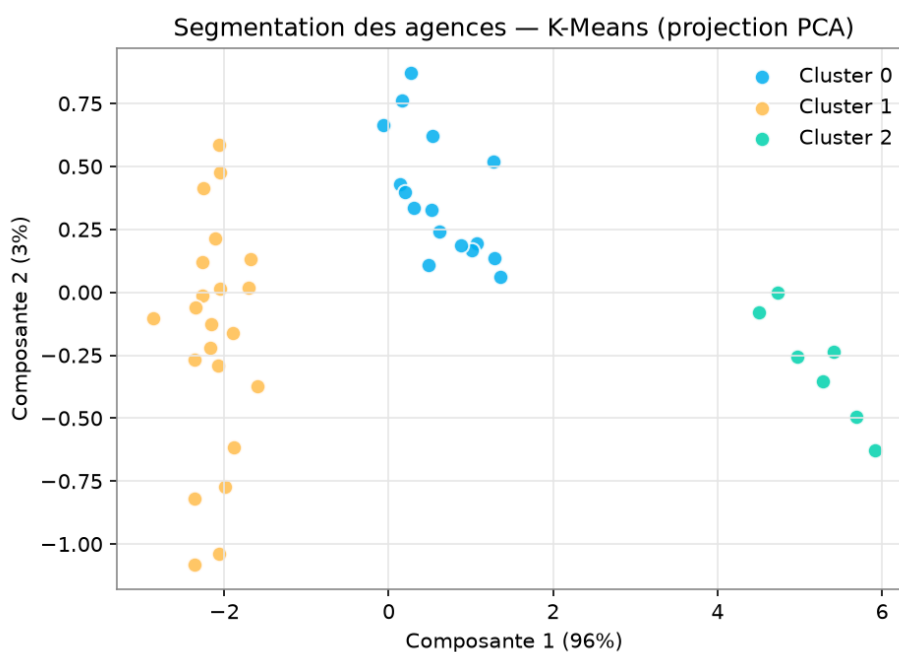


Figure 4.14 – Segmentation des agences (K-Means) projetée par PCA

Le profil moyen de chaque segment (tableau 4.10) permet de les caractériser.

Table 4.10 – Profil moyen des segments d'agences

Segment	Nb	Effectif	Clients	Comptes	Prod. crédits	Présence
Grandes agences	7	116	908	2 333	1 734	0,98
Moyennes agences	16	54	438	1 076	823	0,94
Petites agences	22	21	160	431	321	0,87

On distingue ainsi un segment de **grandes agences** (pôles majeurs à fort effectif et forte production), un segment d'**agences moyennes** (activité intermédiaire) et un segment de **petites agences** (effectif et activité réduits, taux de présence plus faible).

4.7.6 Apport décisionnel

Cette segmentation constitue un outil d'**aide à la décision** : elle permet d'adapter les actions à chaque profil. Le tableau 4.11 décline des recommandations de pilotage par segment.

Table 4.11 – Recommandations de pilotage par segment

Segment	Recommandations
Grandes agences	Capitaliser sur la performance ; fixer des objectifs ambitieux ; en faire des agences de référence et diffuser leurs bonnes pratiques.
Moyennes agences	Fort potentiel de croissance ; renforcer l'effort d'équipement (EER) et la collecte ; assigner des objectifs progressifs.
Petites agences	Accompagnement ciblé ; suivi rapproché de l'assiduité (taux de présence plus faible) ; mutualisation des ressources et montée en compétence.

En résumé, la segmentation permet d'**allouer les ressources** selon la taille, de **différencier les objectifs** et d'**accompagner spécifiquement** les agences les plus fragiles. Elle prolonge le volet décisionnel et illustre l'apport de l'**intelligence artificielle** au pilotage du réseau.

Conclusion

Ce chapitre a doté la solution d'un volet décisionnel complet : une chaîne ETL alimente un datamart consolidé, restitué par le tableau de bord et par Power BI, et exploité par une segmentation des agences qui fait émerger trois profils cohérents et exploitables pour la décision. Le chapitre suivant présente la **réalisation et le déploiement** de l'ensemble de la solution.

Chapitre 5

Réalisation et déploiement

Introduction

Ce dernier chapitre présente la **réalisation** et le **déploiement** de la solution. Il décrit d’abord l’environnement de travail — matériel, logiciels et technologies mobilisés —, illustre au moyen de captures d’écran les principales interfaces développées, puis expose le déploiement de l’application et la validation fonctionnelle.

5.1 Environnement de travail

5.1.1 Environnement matériel

Le développement a été réalisé sur un poste de travail standard : ordinateur portable doté d’un processeur Intel Core i5, de 16 Go de mémoire vive et du système d’exploitation Windows 11.

5.1.2 Outil de rédaction du rapport

Le présent rapport a été rédigé avec **LaTeX**, système de composition de documents particulièrement adapté aux écrits scientifiques et techniques.



Figure 5.1 – Logo de « LaTeX »

5.1.3 Environnement logiciel

Plusieurs outils et technologies ont été mobilisés pour la réalisation du projet.

Java (JDK 21) : langage et plateforme d'exécution sur lesquels repose l'ensemble de l'application.



Figure 5.2 – Logo de « Java (JDK 21) »

Jakarta EE 10 [5] : ensemble de spécifications d'entreprise (Servlets, JSP, EJB, JPA) structurant l'application en couches.



Figure 5.3 – Logo de « Jakarta EE »

Hibernate [9] : implémentation de JPA assurant le mapping objet-relationnel entre les entités Java et la base Oracle.



Figure 5.4 – Logo de « Hibernate »

Oracle Database [6] : système de gestion de base de données relationnelle hébergeant l'ensemble des données opérationnelles et les vues décisionnelles.



Figure 5.5 – Logo d'« Oracle Database »

WildFly / JBoss [15] : serveur d'applications Jakarta EE sur lequel l'application est déployée sous forme d'archive WAR.



Figure 5.6 – Logo de « WildFly / JBoss »

Apache Maven [16] : outil de gestion des dépendances et d'automatisation du *build* (production du WAR).



Figure 5.7 – Logo d'« Apache Maven »

IntelliJ IDEA : environnement de développement intégré utilisé pour le développement, le débogage et le déploiement de l'application.



Figure 5.8 – Logo d'« IntelliJ IDEA »

Git et GitHub : système de gestion de versions et plateforme d'hébergement du code source.



Figure 5.9 – Logos de « Git » et « GitHub »

ApexCharts [17] : bibliothèque JavaScript de visualisation utilisée pour les graphiques du tableau de bord embarqué.



Figure 5.10 – Logo d'« ApexCharts »

Microsoft Power BI [7] : outil de *reporting* décisionnel connecté aux vues analytiques de la base Oracle.



Figure 5.11 – Logo de « Microsoft Power BI »

Python : langage utilisé pour la chaîne **ETL** et la **segmentation** des agences, au moyen des bibliothèques **pandas** (manipulation des données) [12] et **scikit-learn** (apprentissage automatique) [14].



Figure 5.12 – Logos de « Python », « pandas » et « scikit-learn »

5.2 Interfaces de l'application

Cette section illustre les principales interfaces réalisées, présentées dans l'ordre des modules fonctionnels.

5.2.1 Authentification

La figure 5.13 présente la page d'authentification : l'utilisateur y saisit son identifiant et son mot de passe pour accéder à l'application.

5.2.2 Gestion des agences et des employés

La figure 5.14 présente l'interface de gestion des agences : l'administrateur y sélectionne une agence pour en consulter les membres et peut créer, modifier ou supprimer une agence du réseau.

La figure 5.15 montre l'interface de gestion des employés : la liste des employés, les filtres de recherche et le formulaire d'ajout, chaque employé étant rattaché à son agence.

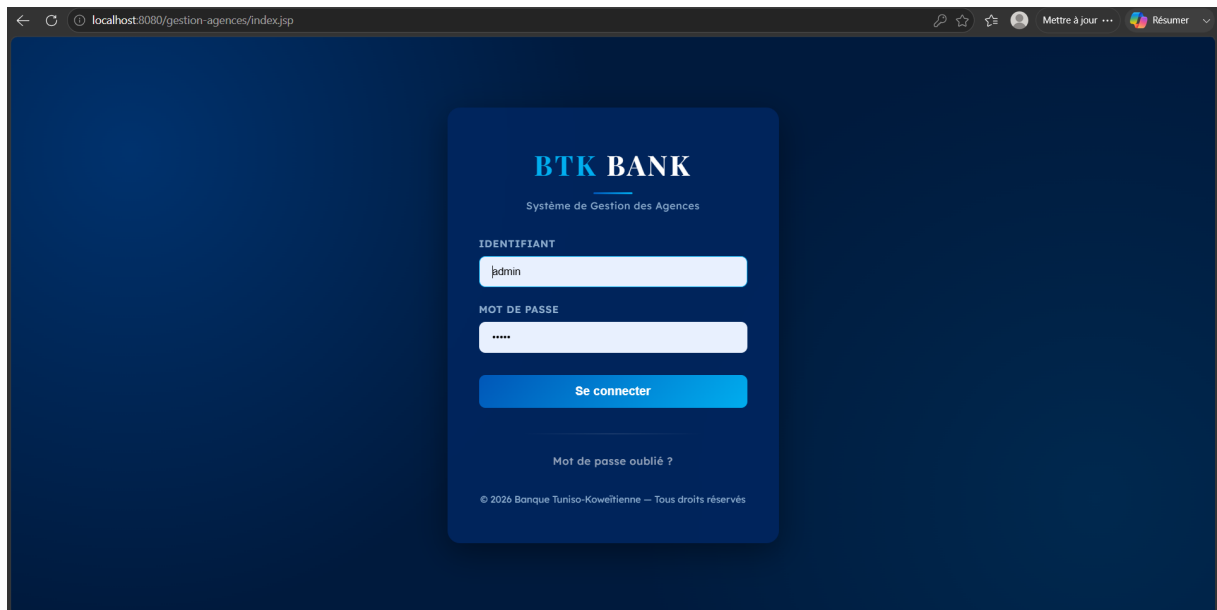


Figure 5.13 – Interface d'authentification

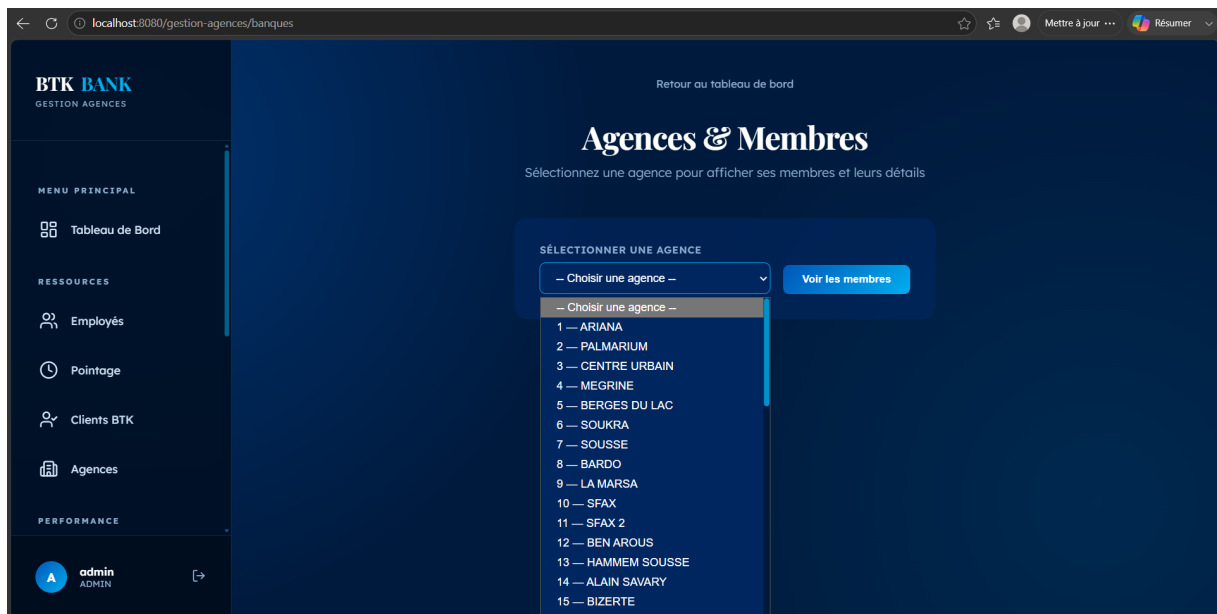


Figure 5.14 – Interface de gestion des agences (sélection et consultation des membres)

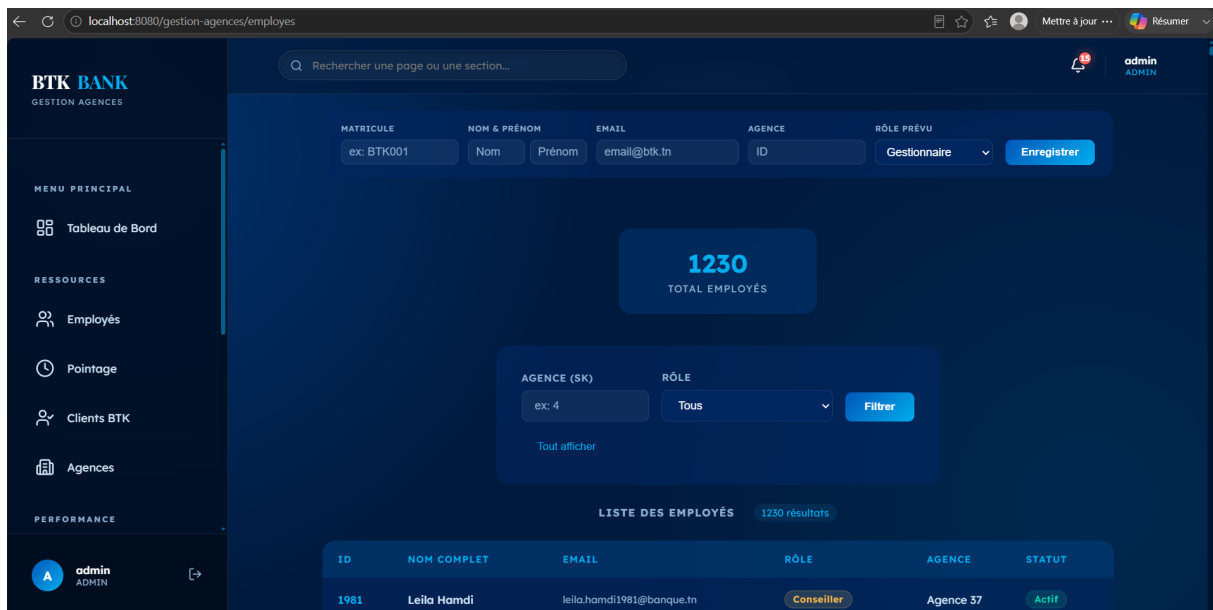


Figure 5.15 – Interface de gestion des employés (ajout, filtres et liste)

5.2.3 Gestion des clients et des objectifs

La figure 5.16 présente l'interface de gestion du portefeuille clients : consultation, recherche et affectation de chaque client à un gestionnaire.

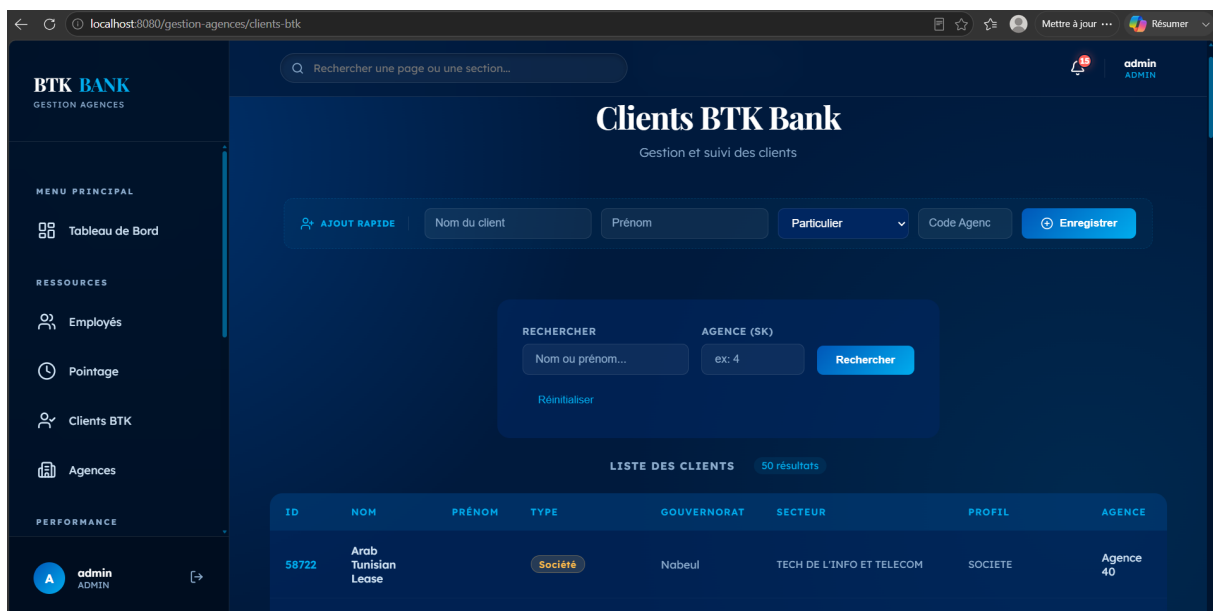


Figure 5.16 – Interface de gestion des clients (portefeuille BTK)

La figure 5.17 montre le suivi des objectifs commerciaux : production de comptes, de crédits et d'épargne restituée par agence et par gestionnaire, pour une période donnée.

Objectifs Commerciaux
Suivi des objectifs par agence et gestionnaire — Total : 376

AGENCE (SK) : ex: 4
GESTIONNAIRE (SK) : ex: 76
Filtrer
Tout afficher

AGENCE	GESTIONNAIRE	SITUATION	EER PART.	EER H. PART.	CPT CHÈQUES	CPT ÉPARGNES	CRÉDITS CONSO	CRÉDITS IMMO	DAV PP
Agence 1	Gest. 67	Detaché	36.0	24.0	27.0	31.0	750000.0	550000.0	144000.0
Agence 1	Gest. 152	CIVP	60.0	24.0	45.0	51.0	900000.0	550000.0	120000.0
Agence 1	Gest. 223	Detaché	96.0	12.0	72.0	82.0	900000.0	550000.0	96000.0
Agence 1	Gest. 307	Permanent	48.0	0.0	36.0	41.0	288000.0	112000.0	48000.0
Agence 1	Gest. 0	Permanent	36.0	0.0	27.0	31.0	0.0	0.0	0.0

Figure 5.17 – Suivi des objectifs commerciaux par agence et gestionnaire

5.2.4 Pointage des employés

La figure 5.18 présente l'interface du module de pointage : les cartes de synthèse (présents, retards, absents, taux de présence), le pointage du jour (boutons « Pointer l'arrivée / le départ ») et le formulaire de saisie ou de correction manuelle.

Pointage des Employés
Présence quotidienne : arrivées, départs et statuts

mercredi 24 juin 2026

0 PRÉSENTS
0 RETARDS
0 ABSENTS
% TAUX DE PRÉSENCE

Mon pointage du jour
Vous n'avez pas encore pointé aujourd'hui.
Pointer l'arrivée
Pointer le départ

Saisie / Correction manuelle
EMPLOYÉ : Leila Hamdi
DATE : jj/mm/aaaa
STATUT : Présent
HEURE D'ARRIVÉE : --:--
HEURE DE DÉPART : --:--
COMMENTAIRE : Optionnel
Enregistrer le pointage

Figure 5.18 – Interface du module de pointage des employés

5.3 Organisation du code

Le projet est structuré en modules distincts, du cœur applicatif Jakarta EE à la chaîne décisionnelle. Le tableau 5.1 récapitule les principaux composants et leur rôle.

Table 5.1 – Principaux modules du projet

Module	Rôle
entity	Entités JPA du domaine (Utilisateur, Agence, Client, Objectif, Pointage...).
servlet	Couche de contrôle (servlets et filtre de sécurité).
service	Couche métier (services applicatifs).
*.jsp	Couche de présentation (interfaces et tableau de bord embarqué).
sql/	Scripts d'initialisation des tables et vues décisionnelles V_BI_*.
etl/	Chaîne ETL en Python (agrégation des indicateurs par agence).
clustering/	Segmentation des agences en Python (scikit-learn).
powerbi/	Thème et rapport décisionnel Power BI.

L'ensemble du code source — application Jakarta EE (entités, servlets, services, pages JSP), scripts SQL d'initialisation, chaîne ETL et segmentation Python, thème et rapport Power BI — est versionné sur le dépôt **Git** associé au projet.

5.4 Déploiement de l'application

L'application est packagée par **Maven** sous forme d'une archive web `gestion-agences.war`, puis déployée sur le serveur d'applications **WildFly**. La persistance repose sur une **unité de persistance JPA** de type JTA, qui accède à la base Oracle via la source de données `java:/OracleDS` déclarée sur le serveur. L'unité de persistance déclare les entités du domaine (Utilisateur, Agence, Client, Objectif, Pointage, Gestionnaire...), le pilote `oracle.jdbc.OracleDriver` et la connexion à la base `FREEPDB1`, ainsi que le dialecte Hibernate pour Oracle.

Le schéma de la base est **géré par les scripts SQL d'initialisation** et non régénéré automatiquement par Hibernate, ce qui garantit la maîtrise de la structure de la base en production.

5.5 Tests et validation

Afin de garantir la conformité de l'application aux besoins spécifiés, des **tests fonctionnels** ont été menés au fil du développement, module par module. Chaque fonctionnalité livrée a été éprouvée à travers des scénarios nominaux et des cas limites (identifiants invalides, accès non autorisé, pointage tardif), puis validée avec l'encadrement. Le tableau 5.2 récapitule les principaux scénarios de test et leur résultat.

Table 5.2 – Principaux scénarios de test fonctionnel

Module	Scénario testé	Résultat
Authentification	Connexion avec des identifiants valides puis invalides.	Conforme
Rôles	Accès à une fonction réservée à l'ADMIN par un USER.	Conforme
Agences	Création, modification et suppression d'une agence.	Conforme
Employés	Ajout d'un employé et rattachement à une agence.	Conforme
Clients	Affectation d'un client à un gestionnaire.	Conforme
Objectifs	Consultation de la production par agence et période.	Conforme
Pointage	Pointage au-delà de 9 h (détermination du statut).	Conforme (RETARD)
Demandes	Validation puis refus d'une demande avec journalisation.	Conforme
ETL	Exécution du pipeline et production du datamart.	Conforme
Décisionnel	Filtrage par district dans le rapport Power BI.	Conforme

L'ensemble des scénarios a produit le comportement attendu, ce qui confirme la **conformité fonctionnelle** de la solution aux besoins exprimés au chapitre 2.

Conclusion

Ce chapitre a présenté l'environnement de travail, les interfaces réalisées, le déploiement de l'application et la validation fonctionnelle. Il clôt la partie technique du projet : la solution, de la gestion opérationnelle jusqu'au volet décisionnel, est désormais réalisée, déployée et opérationnelle.

Conclusion générale et perspectives

Ce projet de fin d'études a permis de concevoir et de réaliser, au sein de la **BTK – Banque Tuniso-Koweïtienne**, une application web de gestion des agences bancaires adossée à un volet décisionnel, conduite selon la méthodologie **CRISP-DM**. La solution centralise la gestion des agences, des employés, des clients et des objectifs, assure le **pointage** des employés, encadre les opérations sensibles par des circuits de validation et restitue des indicateurs d'aide à la décision via un tableau de bord et Power BI. Elle intègre en outre une chaîne **ETL** et une brique d'**intelligence artificielle** : la segmentation des agences par apprentissage non supervisé (clustering).

Ce travail a été l'occasion de mettre en pratique les compétences de la formation — modélisation UML, développement Jakarta EE, base de données Oracle, chaîne décisionnelle (ETL en Python et *reporting* Power BI) et initiation à l'**apprentissage automatique** — au croisement du développement logiciel et de la Business Intelligence.

Plusieurs **difficultés** ont jalonné le projet et ont constitué autant d'occasions d'apprentissage. La **consolidation** de données dispersées à des grains hétérogènes a nécessité une chaîne ETL robuste et rejouable. L'articulation entre le système opérationnel Oracle et l'outil de restitution Power BI a demandé une attention particulière à la **modélisation décisionnelle** (schéma en étoile, mesures) et à la préparation des données. Enfin, la mise au point de la segmentation a exigé une démarche méthodique de choix et de comparaison de modèles afin d'en garantir la robustesse. Ces défis ont renforcé une compétence clé : **transformer une donnée brute en information utile à la décision**.

Plusieurs perspectives d'évolution peuvent être envisagées :

- le chiffrement (hachage) des mots de passe et le renforcement de la sécurité ;
- l'**industrialisation** de la chaîne ETL : ordonnancement automatique et historisation incrémentale des indicateurs ;
- l'enrichissement par des analyses prédictives (prévision des objectifs) ;
- l'exposition d'une API REST pour l'interopérabilité avec d'autres systèmes ;
- le développement d'une application mobile de consultation des indicateurs.

Bibliographie

- [1] BTK Bank, "Banque tuniso-koweïtienne – site officiel," 2025. <https://www.btknet.com/>.
- [2] K. Schwaber and J. Sutherland, "The scrum guide : The definitive guide to scrum," 2020. <https://scrumguides.org/>.
- [3] P. Chapman, J. Clinton, R. Kerber, T. Khabaza, T. Reinartz, C. Shearer, and R. Wirth, "Crisp-dm 1.0 : Step-by-step data mining guide," tech. rep., SPSS Inc., 2000.
- [4] P. Roques, *UML 2 par la pratique : études de cas et exercices corrigés*. Eyrolles, 2018.
- [5] Eclipse Foundation, "Jakarta ee 10 – documentation officielle," 2025. <https://jakarta.ee/>.
- [6] Oracle Corporation, "Oracle database – documentation," 2025. <https://docs.oracle.com/en/database/>.
- [7] Microsoft, "Microsoft power bi – documentation," 2025. <https://learn.microsoft.com/power-bi/>.
- [8] A. Goncalves, *Beginning Java EE 7*. Apress, 2013.
- [9] Red Hat, "Hibernate orm – documentation," 2025. <https://hibernate.org/orm/documentation/>.
- [10] M. Fowler, *UML Distilled : A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley Professional, 3 ed., 2004.
- [11] R. Kimball and M. Ross, *The Data Warehouse Toolkit : The Definitive Guide to Dimensional Modeling*. John Wiley & Sons, 3 ed., 2013.
- [12] The pandas development team, "pandas – python data analysis library," 2025. <https://pandas.pydata.org/docs/>.
- [13] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 2 ed., 2019.
- [14] scikit-learn developers, "scikit-learn – machine learning in python," 2025. <https://scikit-learn.org/stable/>.
- [15] Red Hat, "Wildfly application server – documentation," 2025. <https://docs.wildfly.org/>.

- [16] Apache Software Foundation, "Apache maven – documentation," 2025. <https://maven.apache.org/guides/>.
- [17] ApexCharts, "Apexcharts.js – documentation," 2025. <https://apexcharts.com/docs/>.